Type-Directed Discretization of Probabilistic Programs

KATHERINE WU, Cornell University, USA
JULES JACOBS, Jane Street, USA
KEVIN BATZ, Cornell University, USA
ALEXANDRA SILVA, Cornell University, USA

We study *exact discretization* as a semantics-preserving transformation for recursive, higher-order probabilistic programs with continuous distributions. We target programs where continuous values are compared against finitely many constants, so exact inference reduces to a discrete problem. Our central technical contribution is a non-local, type-directed analysis that infers where continuous values can be partitioned into finitely many observationally relevant regions, then rewrites sampling and comparison behavior over those regions. We call this transformation Slice. Because this construction is global and type-directed, correctness requires reasoning beyond the local syntax: we formalize the transformation and prove soundness for boolean queries using a coupling-style logical relations argument over operational semantics. intended coupling argument and isolate the measurable weak-simulation and terminal-limit obligations needed for soundness. Constructing that weak simulation remains open. As an application, transformed programs can be executed by discrete engines such as Dice, Roulette, and Storm. Our empirical evaluation shows two complementary strengths of Slice when paired with discrete backends: it enables *exact inference* for case studies combining continuous sampling, higher-order functions, and unbounded recursion, and, on benchmarks where direct comparison is possible, it is competitive with state-of-the-art exact inference systems for continuous programs.

CCS Concepts: • **TODO**;

Additional Key Words and Phrases: probabilistic programming, exact inference, continuous distributions, discretization, type systems

ACM Reference Format:
Katherine Wu, Jules Jacobs, Kevin Batz, and Alexandra Silva. 2026. Type-Directed Discretization of Probabilistic Programs. 1, 1 (August 2026), 117 pages. https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

1 Introduction

Probabilistic programming languages (PPLs) are widely used to model uncertainty [6, 10, 14, 15, 17, 21, 24, 25, 27, 34, 35, 37, 39–41, 45–47]. Programs with *continuous* distributions capture rich models but require reasoning about uncountable domains, whereas purely *discrete* programs often admit fast and precise inference but cannot express real-valued models.

Slice studies *exact discretization*: a semantics-preserving source-to-source transformation that converts, if possible, continuous programs into discrete ones. We target higher-order recursive programs with conditioning and continuous sampling, without arithmetic. The key mechanism is *type-directed cut inference*: our type system tracks each constant threshold (or *cut*) used in comparisons and uses this information to partition continuous domains into finitely many intervals. For example, $\texttt{uniform}(0,1) < 0.5$ discretizes to $\texttt{discrete}(0.5, 0.5) < 1$, yielding a discrete program that existing exact inference tools can process.

Cuts may arise far from their source expressions: a sampled value may flow through higher-order functions, conditionals, recursive calls, data structures, and observations before it is eventually compared. To discretize soundly, Slice propagates cut information along these non-local flows, collecting all observationally relevant cut points for each continuous value and using them to partition its domain into finitely many intervals (see §2 for illustrative examples). A type system provides a natural framework for this analysis: it gives a compositional account of how cut information propagates through each language construct (via one typing

Authors' Contact Information: Katherine Wu, kaw324@cornell.edu, Cornell University, Ithaca, New York, USA; Jules Jacobs, julesjacobs@gmail.com, Jane Street, New York, New York, USA; Kevin Batz, ksb239@cornell.edu, Cornell University, Ithaca, New York, USA; Alexandra Silva, alexandra.silva@cornell.edu, Cornell University, Ithaca, New York, USA.

2026. ACM XXXX-XXXX/2026/8-ART
https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

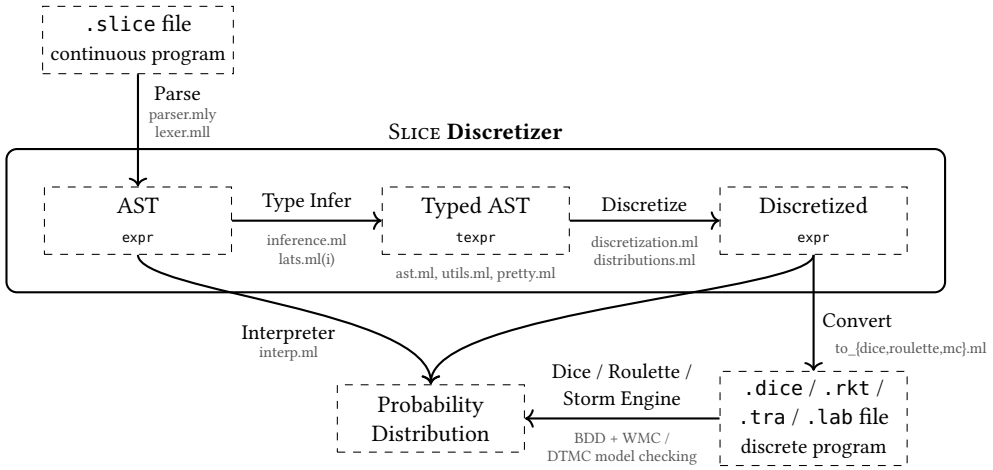


Fig. 1. Pipeline for exact discretization: typed Slice programs are transformed into semantically equivalent discrete programs, then evaluated by discrete engines. The transformation stage (type inference + discretization) is backend-agnostic; Dice, Roulette, and Storm are examples of target backends.

rule per langauge feature); it separates the analysis result, i.e. a type derivation, from the type inference algorithm used to infer it; and it supports our correctness argument, since semantic preservation can be stated as a logical relation indexed by our types carrying cut and value sets.

Our discretization method is subject to a principled restriction: a continuous value must ultimately be compared against constants; direct comparisons between two unknown reals such as $\texttt{uniform}(0,1) < \texttt{uniform}(0,1)$ cannot be discretized exactly and remain continuous. Nonetheless, benchmarks from the literature and our novel case studies show that many programs *can* be discretized by our system automatically.

Discretization itself is not new. Garg et al. [20] encode real-valued variables via user-chosen bit-precisions, trading accuracy for tractability. Other approximate discretization schemes exist in the literature [5, 12, 28]. In Slice, *exact discretization* means that the transformation is semantics-preserving: the discrete program preserves names the specification that the discrete program preserve the relevant probabilities of the original continuous program. This differs from approximate discretization, which approximates the original semantics and may provide error bounds or convergence guarantees, but does not prove equality of source and transformed program probabilities in the sense of our main theorem. Our contextual-equivalence theorem is conditional on the weak-simulation and terminal-limit obligations made explicit in §6.

Figure 1 shows the concrete pipeline from typed source programs to discrete backends. Because discretization is source-to-source, any discrete PPL that supports finite distributions can serve as a backend; we instantiate this with Dice [27], Roulette [35], and Storm [16].

Outline. Starting from examples (§2) and concluding with related and future work in §8, we:

Type System (§3) Characterize when programs are discretizable.

Type Inference (§4) Infer types automatically to guide discretization.

Exact Discretization (§5) Define a type-directed transformation from continuous programs to discrete ones using inferred type information.

Soundness (§6) Prove semantic preservation of the discretization transformation with a coupling-style relational argument [7, 13, 48].

Implementation, Case Studies, and Evaluation (§7) Implement Slice type inference and the discretization transformation in OCaml, and provide Dice, Roulette, and Storm backends for running the discretized programs. We provide (i) case studies combining continuous sampling, higher-order functions, and unbounded recursion; to the best of our knowledge, there is no prior exact inference system that can cover

$e : \tau$	Condition	$\mathcal{D}[\![e : \tau]\!]$
$x : \tau$		x
$c : \texttt{float}[B;V]$	$B = \top$	c
	$B \neq \top$	$k_{\#(\lvert B\rvert+1)}$ such that $c \in \text{Intervals}(B)_k$
$\texttt{let}\ x = e_1 : \tau_1\ \texttt{in}\ e_2 : \tau_2 : \tau_2$		$\texttt{let}\ x = \mathcal{D}[\![e_1 : \tau_1]\!]\ \texttt{in}\ \mathcal{D}[\![e_2 : \tau_2]\!]$
$(\texttt{if}\ e_1 : \texttt{bool}\ \texttt{then}\ e_2 : \tau\ \texttt{else}\ e_3 : \tau) : \tau$		$\texttt{if}\ \mathcal{D}[\![e_1 : \texttt{bool}]\!]\ \texttt{then}\ \mathcal{D}[\![e_2 : \tau]\!]\ \texttt{else}\ \mathcal{D}[\![e_3 : \tau]\!]$
$(e_1 : \texttt{float}[B;V_1] < e_2 : \texttt{float}[B;V_2]) : \texttt{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B;V_1]]\!] <_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \texttt{float}[B;V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B;V_1]]\!] < \mathcal{D}[\![e_2 : \texttt{float}[B;V_2]]\!]$
$(e_1 : \texttt{float}[B;V_1] \leq e_2 : \texttt{float}[B;V_2]) : \texttt{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B;V_1]]\!] \leq_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \texttt{float}[B;V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B;V_1]]\!] \leq \mathcal{D}[\![e_2 : \texttt{float}[B;V_2]]\!]$
$\texttt{uniform}(e_1 : \texttt{float}[B_1;V_1],\ e_2 : \texttt{float}[B_2;V_2]) : \texttt{float}[B;V]$	$e_1, e_2 \in \mathbb{R},\ e_1 > e_2$	$\texttt{diverge} : \tau$
	$B = \top$	$\texttt{uniform}(\mathcal{D}[\![e_1 : \texttt{float}[B_1;V_1]]\!], \mathcal{D}[\![e_2 : \texttt{float}[B_2;V_2]]\!])$
	$e_1, e_2 \in \mathbb{R}$, $B = \{\sim_1 b_1, \ldots, \sim_n b_n\}$	$\texttt{discrete}(p_0, \ldots, p_n)$ where $p_k = \text{CDF}(b_{k+1}) - \text{CDF}(b_k)$
	$B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, $V_1 = \{v_1, \ldots, v_n\}$, $V_2 = \{w_1, \ldots, w_m\}$	$\texttt{let}\ v_1' = \mathcal{D}[\![e_1 : \texttt{float}[B_1;V_1]]\!]\ \texttt{in}$ $\texttt{let}\ v_2' = \mathcal{D}[\![e_2 : \texttt{float}[B_2;V_2]]\!]\ \texttt{in}$ $\texttt{if}\ v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_1 : \texttt{float}[B_1;V_1]]\!]$ $\quad \&\&\ v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_1 : \texttt{float}[B_2;V_2]]\!]\ \texttt{then}$ $\quad\quad \mathcal{D}[\![\texttt{uniform}(v_1, w_1) : \texttt{float}[B;V]]\!]$ $\ldots$ $\texttt{else if}\ v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_i : \texttt{float}[B_1;V_1]]\!]$ $\quad \&\&\ v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_j : \texttt{float}[B_2;V_2]]\!]\ \texttt{then}$ $\quad\quad \mathcal{D}[\![\texttt{uniform}(v_i, w_j) : \texttt{float}[B;V]]\!]$ $\ldots$ $\texttt{else}\ \mathcal{D}[\![\texttt{uniform}(v_n, w_m) : \texttt{float}[B;V]]\!]$

Fig. 13. Discretization function, where $\text{CDF}(b_0) = 0$ and $\text{CDF}(b_{n+1}) = 1$. For the **uniform** distribution, $V = \top$ is implicit, and all remaining cut-set and value-set cases are ruled out by the constraints enforced during type inference. For all other language constructs, $\mathcal{D}[\![\cdot]\!]$ recurses structurally over them.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression $e : \texttt{bool}$ and its type annotations, a *discretized* expression $\mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool}$ such that e and $\mathcal{D}[\![e : \texttt{bool}]\!]$ are *semantically equivalent*, i.e., they induce the same distribution. We refer to this intended semantics-preserving transformation as *exact discretization*, to distinguish it from approaches [20, 28] that approximate continuous programs via externally chosen precision parameters. Exactness here is the equality proved by Theorem 6.1: the transformed program preserves the relevant probabilities of the source program, not merely an approximation with an error boundis stated conditionally in Theorem 6.1; constructing the required measurable weak simulation and proving its terminal-limit compatibility remain open obligations. Intuitively, when a program uses continuous quantities only through finitely many comparison outcomes, exact inference should reduce to a discrete problem, and exact discretization makes that reduction explicit.

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations; we focus on continuous uniform distributions, with others handled analogously. The only sub-expressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information; §6 proves that this non-local, type-mediated construction preserves semantics. Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form $e = \texttt{uniform}(e_1, e_2)$ of type $\texttt{float}[B;V]$ with $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$ can be *discretized* to an expression $e' : \texttt{fin}(n+1)$ such that the probability

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \mathbf{fin}(|B|+1) & \text{if } \tau = \mathbf{float}[B;V] \text{ and } B \neq \top \\ \mathbf{float}[B;V] & \text{if } \tau = \mathbf{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

The continuous $\mathbf{uniform}(0,x)$-expression is replaced by a case distinction on the discretizations of $\mathbf{uniform}(0,10)$ and $\mathbf{uniform}(0,20)$, with each case executed with the correct probability 0.5. Since this reduces the remaining discretization process to discretizing expressions with *constant parameters only*, the remaining steps are completely analogous to Theorem 5.1.

Crucially, the *recoverability* constraints in the Continuous-rules from Figure 6 ensure that the parameter expressions of the respective sampling expressions can take only finitely many values, and that the cut sets are fine-grained enough to recover the values the parameter expressions evaluate to, as demonstrated in the preceding example.

A similar intuition applies to discretizable comparisons $e_1 < e_2$, where *both* expressions are non-constant but finite-cut. The *answers* constraints in the rule Less (resp. Less-Equal) ensure that at least one of e_1 and e_2 evaluates to only finitely many values, which enables discretization:

Example 5.3. Consider the following typed expression $(e_1 \leq e_2) : \mathbf{bool}$, given by

```
(if discrete(0.5, 0.5) <#2 1#2 then
       uniform(0,10) : float[B; T] else 2.5 : float[B; {2.5}])
<=
(if discrete(0.5, 0.5) <#2 1#2
       then 2.5 : float[B; {2.5}] else 3.5 : float[B; {3.5}])
```

where $B = \{\leq 2.5, \leq 3.5\}$, and where e_1 : `float[B; T]` and e_2 : `float[B; {2.5,3.5}]`. Intuitively, e_1 samples from a mixture of a uniform and a dirac distribution, and e_2 samples from a finite distribution. Why can $e_1 \leq e_2$ be discretized even though neither e_1 nor e_2 is a constant? As suggested by its type, e_2 can take only finitely many values, namely $\{2.5, 3.5\}$. We apply $\mathcal{D}[\![\cdot]\!]$:

```
(if discrete(0.5, 0.5) <#2 1#2 then discrete(0.25, 0.1, 0.65) else 0#3)
<=
(if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3)
```

Notice that the probability of e_1 evaluating to some value in $(-\infty, 2.5]$ indeed *coincides* with the probability of the discretization of e_1 evaluating to $0_{\#3}$. For e_2, on the other hand, the probability of it evaluating to 2.5 (resp. 3.5) *coincides* with the probability of its discretization evaluating to $0_{\#3}$ (resp. $1_{\#3}$). For this reason, we obtain a semantically equivalent discretization of $e_1 \leq e_2$.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

Theorem 5.4 (Discretization Is Well-Typed). *Let* $\Gamma \vdash e : \tau$. *We have* $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, *where* $\mathcal{D}[\![\Gamma]\!]$ *transforms all types in* Γ, *i.e.,* $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ *for all* $x \in \mathrm{dom}(\Gamma)$.

6 Operational Semantics and Soundness

We now establish the soundness of our approach in the following sense: For *every* expression e : **bool**, possibly including *continuous sampling, higher-order functions, recursion, and conditioning*, The intended soundness statement concerns the *uniform fragment*: the syntax in Figure 5 without sequencing, non-strict comparisons, finite comparisons in source programs, or continuous samplers other than **uniform**. Write $Expr_\tau^{\mathsf{U}}$ for its well-typed expressions and $Expr_\tau^{\mathsf{U}+}$ for the generated-target operational closure. The program-level statement is restricted to closed e : $\mathbf{bool} \in Expr_{\mathbf{bool}}^{\mathsf{U}}$:

e : **bool** *and* $\mathcal{D}[\![e : \textbf{bool}]\!]$: **bool** *(obtained from Figure 13) are semantically equivalent.*

The first step is to make formal sense of semantic equivalence. For that, we will define a monadic operational semantics for our language in §6.1. We will then proceed by proving the above claim . This is non-trivial: discretization A complete proof of the displayed claim additionally requires the measurable weak-coupling obligations stated below. Discretization decisions are driven by non-local, type-propagated information, so we cannot proceed by a local rule-by-rule argument. To prove the claim rigorously, we relate the executions of e and $\mathcal{D}[\![e]\!]$ through a coupling-style logical relation in §6.1. and the literal generated target takes administrative steps that do not synchronize with source steps.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics has to distinguish between non-termination and observation failure, which we realize as follows: The "missing" probability mass in $\langle e : \tau\rangle$ is the probability that e diverges. The probability of $\bot$ is the probability of encountering an observation failure. With this model, semantic equivalence indeed captures our sought-after correctness claim: If $\langle e : \textbf{bool}\rangle = \langle \mathcal{D}[\![e : \textbf{bool}]\!] : \textbf{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!]\colon Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

Let $\tau = \textbf{bool}$ for the sake of concreteness. Care must be taken since, e.g., sub-expressions of type $\textbf{float}[B; \{3.5, 4.5\}]$ cannot evaluate to *arbitrary* reals but are restricted to the values $\{3.5, 4.5\}$. We tackle this by extracting from each type-annotated expression a suitable *skeleton*, where float constants are replaced by "holes" (hence the term "skeleton"). For instance, the expression

$$e \quad = \quad (x : \textbf{float}[\{<3.5, <4.5\}; \top] \;<\; y : \textbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \textbf{bool}$$

gives rise to the skeleton

$$s \quad = \quad (\square : \textbf{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \textbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \textbf{bool}$$

and the type annotations tell us that the possible values these holes can be filled with are in $\mathbb{R} \times \{3.5, 4.5\}$. We can thus inherit a measurable space over all well-typed expressions arising from this particular skeleton s from the standard Borel measurable space over $\mathbb{R} \times \{3.5, 4.5\}$. The measurable space over *all* well-typed expressions of type **bool** (i.e., over $Expr_{\textbf{bool}}$) is then obtained from the measurable spaces of *all* skeletons that give rise to expressions of type **bool**.

Equipped with this family of measurable spaces, we define the small-step functional $[\![\cdot]\!]$ in the expected, type-preserving manner (cf. Tables 1 and 2). For instance, if e_1 is not a value, then[1]

$$[\![(\textbf{let}\ x = e_1 : \tau_1\ \textbf{in}\ e_2 : \tau_2) : \tau_2]\!] \;=\; \underbrace{[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{(\textbf{let}\ x=g\ \textbf{in}\ e_2:\tau_2):\tau_2}}_{\text{strict sub-Markov bind with an ordinary-input continuation}} \quad ,$$

i.e., $[\![e_1 : \tau_1]\!]$ yields the sub-distribution over expressions obtained from executing e_1 for *one step*, and the monadic bind $\gg\!=$ "folds" this distribution into the surrounding **let**-statement. The intuition for the remaining statements is analogous. Crucially, $\gg\!=$ propagates the probability of encountering $\bot$ — an observation failure — through executions. For instance, assume that e_1 from above is given by **observe** (false : **bool**) : **unit**. Since e_1 encounters an observation failure with probability 1, i.e., $[\![\textbf{observe}\ (\text{false} : \textbf{bool}) : \textbf{unit}]\!](\{\bot\}) = 1$, the same holds for the surrounding **let**-statement, i.e., also $[\![(\textbf{let}\ x = e_1 : \tau_1\ \textbf{in}\ e_2 : \tau_2) : \tau_2]\!](\{\bot\}) = 1$. All details — including proofs that all monadic binds are well-behaved since they operate on suitable Markov kernels

[1]Put formally, $(\mu \gg\!= f)(A) = \int f(x)(A)\cdot\mu(dx)$for a sub-Markov kernel $f : X \to \mathcal{D}(Y + \bot)$, $(\mu \gg\!= f)(A) = \int_{X+\bot} f_\bot(x)(A)\,\mu(dx)$, where $f_\bot(\bot) = \delta_\bot$; see Definition F.28for details. δ_e denotes the Dirac distribution of e.

— are provided in §F.4The appendix defines this strict bind by extending its ordinary-input continuation canonically at $\bot$. The remaining measurability obligations for the small-step kernel are stated explicitly there.

6.1.2 From Small-Step to Big-Step Semantics. We now define $\langle\cdot\rangle$ via $[\![\cdot]\!]$. For that, we first lift the one-step semantics $[\![\cdot]\!]$ to an n-step semantics $\langle\cdot\rangle_n$ in the expected way:

$$\langle e : \tau\rangle_0 = \delta_{e:\tau} \qquad \text{and} \qquad \langle e : \tau\rangle_{n+1} \;=\; \langle e : \tau\rangle_n \gg\!= \lambda e'.\, \begin{cases} \delta_\bot & \text{if } e' = \bot \\ [\![e' : \tau]\!] & \text{otherwise} \end{cases}$$

This construction is well-defined since the functional $[\![\cdot]\!]$ itself gives rise to a suitable Markov kernel(Theorem F.39). We then define Conditional on Theorem F.39, this construction uses a suitable sub-Markov kernel. The ordinary terminal-limit construction and its countable-additivity proof are supplied by the separate terminal-limit repair. It defines for every type τ the functional $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$ as by $\langle e : \tau\rangle = \lambda A.\, \lim_{n\to\infty} \langle e : \tau\rangle_n(A)$, which is well-defined by the monotone convergence theorem: the sequence $(\langle e : \tau\rangle_n)_{n\in\mathbb{N}}$ is monotonically increasing for every measurable set where A over $Vals_\tau + \bot$ (cf. Lemma F.48) and upper-bounded by 1: executing a program for more and more steps can only increase the probability of terminating in a given set of values or an observation failureranges over the terminal subspace $Vals_\tau + \bot$. The terminal restrictions, rather than the full expression-space measures, increase eventwise. This branch relies on that separate proof and does not claim that the full finite-step measures are monotone.

6.2 Soundness via Probabilistic Couplings

We first state the central theorem, then outline our coupling-style correctness argumentstate the intended result conditionally on the weak-coupling obligations that remain to be proved.

Theorem 6.1 (Conditional closed Boolean outcome preservation). *Let e : **bool**. Then $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$. Under Definitions F.41 and F.50, if $e : \mathbf{bool} \in Expr^{\cup}_{\mathbf{bool}}$ is closed, then $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$.*

Intuitively, we prove that the two programs can execute in a synchronized manner such that their probabilities of terminating in value v coincide for each $v \in \{\text{true}, \text{false}, \bot\}$The exact graph $P_\tau = \{(e, \mathcal{D}[\![e]\!]) \mid e \in Expr^{\cup}_\tau\}$ is a useful initial carrier, but it is not closed under the displayed synchronous rules. We instead assume a measurable weak relation R_τ and a sub-Markov coupling kernel whose error-preserving pushforwards are the source step and a target weak step, as specified in Definition F.41. The sub-Markov kernel machinery is supplied by the separate kernel-category repair.

Towards this end, define for every type τ the set $P_\tau = \{(e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]) \mid e : \tau\}$. We define a *coupled small-step semantics* $[\![\cdot, \cdot]\!] : P_\tau \to \mathcal{D}(P_\tau + \bot)$ (cf. Tables 3 and 4), which models the simultaneous execution of e and $\mathcal{D}[\![e : \tau]\!]$. The key property of this coupled semantics is that the semantics of both e and $\mathcal{D}[\![e : \tau]\!]$ can be recovered in a sense we formalize next.

Given $\mu \in \mathcal{D}(P_\tau + \bot)$, we define the *projection of μ onto the i-th component* for $i \in \{1, 2\}$ as With projections, we recover the semantics of e and $\mathcal{D}[\![e : \tau]\!]$ as follows:

Lemma 6.2 (Conditional small-step projections). *Let $e : \tau$. We have: Under Definition F.41, for every $(e, t) \in R_\tau$,*

(1) $\pi_1([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![e : \tau]\!]\pi_1(K_\tau(e, t)) = [\![e : \tau]\!]$.

(2) $\pi_2([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![\mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]\pi_2(K_\tau(e, t))$ is the target weak-step distribution from t.

Now, using a limit construction analogous to §6.1.2 for obtaining a coupled *big-step* semantics $\langle\cdot,\cdot\rangle : P_\tau \to \mathcal{D}(V_\tau + \bot)$, where $V_\tau = \{(v : \tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau \text{ is a value}\}$, we prove a suitable continuity property of projection (cf. Lemma F.52) to conclude that Lemma 6.2 transfers to $\langle\cdot,\cdot\rangle$. Finally, observing that $V_{\texttt{bool}} + \bot = \{(\text{true}, \text{true}), (\text{false}, \text{false}), \bot\}$ yields the desired claim: The weak-step interface alone does not imply a coupled terminal limit. The additional premise Definition F.50 requires increasing terminal restrictions whose component limits are the two ordinary big-step semantics. It also requires related Boolean values to have the same truth value. Under those premises, the two projections assign equal mass to true, false, and $\bot$, which proves Theorem 6.1. Constructing the weak relation and proving the terminal-limit premise remain open obligations.

Proof of Theorem 6.1. Let $v \in \{\text{true}, \text{false}, \bot\}$. We have

7 Implementation, Case Studies, and Evaluation

We have implemented the Slice type inference and discretization procedures in OCaml. The implementation is open-source and available at https://github.com/[anonymized], together with all benchmarks, scripts, and a Docker image for reproduction. To compute the interval masses of continuous distributions, we use the OCaml GSL package [19] for cumulative distribution function calculations. All experiments were run in this Docker image on an Apple M3 with 16 GB of RAM.

This section is driven by two research questions:

(1) Does Slice enable exact inference on programs combining continuous sampling, higher-order functions, and unbounded recursion?
(2) On programs that are within the scope of existing tools for exact inference, is the combination of Slice and discrete inference back-ends competitive?

For the first question, we provide a collection of case studies that involve continuous distributions together with recursion, higher-order functions, lists, or conditioning in §7.1. For these case studies, we use Slice to transform the original programs to finite discrete models and analyze the resulting program with Storm [16] — a probabilistic model checker.

For the second question, we evaluate Slice+Dice and Slice+Roulette on two classes of benchmarks in Sections 7.2 and 7.3: (i) synthetic scaling benchmarks that let us vary program size and dependency structure in a controlled way, and (ii) benchmark suites from prior work [22, 40]. We compare against Sppl [40], an exact inference engine supporting continuous probabilistic programs, on benchmarks that *both* Slice and Sppl support, and we additionally report Psi and Hakaru timings for the representative prior-work benchmarks. We remark, however, that Sppl supports arithmetic, as well as (soft) conditioning on measure zero events, which Slice does not support. Our evaluation merely serves the purpose of investigating (i) the overhead of discretization and (ii) the benefits of being able to use efficient *BDD-based* inference back-ends (i.e., Dice and Roulette) for exact inference on *continuous programs*.

7.1 Slice+Storm: Case Studies

There is no exact inference system that, to our knowledge, can cover programs combining all three features: continuous sampling, higher-order functions, and unbounded recursion. As a consequence, there is no exact inference engine for the programs in Sections 2.6, 7.1.2 and 7.1.3. Instead, we use *probabilistic model checking*: The small-step semantics of our language (cf. Tables 1 and 2) for discrete programs gives rise to a *Markov chain* modeling the stochastic process of program execution, inspired by [2] on imperative programs. If that Markov chain is finite, we can use the probabilistic model checker Storm [16] for exact inference. Let us start with an illustrative example.

7.1.1 Conditional Resampling Loop. Consider the continuous program depicted in Figure 15, containing continuous sampling, unbounded recursion, and conditioning inside of the recursion. Slice automatically discretizes this program, also shown in Figure 15. We generate the corresponding Markov chain by unfolding the small-step semantics in a breadth-first manner. In the presence of unbounded recursion, this Markov chain *might* be infinite and this process might thus diverge. If, however, it terminates, the resulting Markov chain is finite and precisely captures the execution behavior of the discretized program. For this example, this is

Measure. Let (X, Σ) be a measurable space. A function $\mu : \Sigma \to [0, \infty]$ is called a measure if it satisfies the following properties: nonnegativity, $\mu(\emptyset) = 0$, and countable additivity. When Σ is a Borel σ-algebra, μ is called a Borel measure.

Measure space. The triple (X, Σ, μ) is called a measure space. When Σ is the Borel σ-algebra and μ is a Borel measure, (X, Σ, μ) is called a Borel measure space.

Subprobability measure. Let (X, Σ, μ) be a measure space. When $\mu(X) \leq 1$, μ is called a (Borel) subprobability measure. When $\mu(X) = 1$, μ is called a (Borel) probability measure and (X, Σ, μ) a (Borel) probability space.

Product measurable space. The product of two measurable spaces (X_1, Σ_1) and (X_2, Σ_2) is also a measurable space, namely $(X_1 \times X_2, \Sigma_1 \times \Sigma_2)$, where $\Sigma_1 \times \Sigma_2$ $(X_1 \times X_2, \Sigma_1 \otimes \Sigma_2)$, where $\Sigma_1 \otimes \Sigma_2$ is the σ-algebra generated by sets of the form $A_1 \times A_2$ with $A_1 \in \Sigma_1$ and $A_2 \in \Sigma_2$.

Generated σ-algebra. Let X be a set and let $\mathcal{G} \subseteq \mathcal{P}(X)$ be a collection of subsets of X. The σ-algebra generated by $\mathcal{G}$, denoted $\sigma(\mathcal{G})$, is the smallest σ-algebra on X that contains $\mathcal{G}$. Equivalently, it is the intersection of all σ-algebras on X that contain $\mathcal{G}$.

Subprobability distribution. Let (X, Σ) be a measurable space. We write $\mathcal{D}(X)$ for the set of all subprobability measures on (X, Σ).

Dirac measure. Let (X, Σ) be a measurable space and let $x \in X$. The *Dirac measure* at x is the probability measure

$$\delta_x : \Sigma \to [0, 1]$$

defined by

$$\delta_x(A) = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{if } x \notin A. \end{cases}$$

Measurable function. A function $f \colon X \to Y$ between measurable spaces $(X, \mathcal{F}_X)$ and $(Y, \mathcal{F}_Y)$ is called measurable if the preimage of any measurable set in Y is measurable in X, i.e. if

$$f^{-1}(A) = \{x \in X \mid f(x) \in A\} \in \mathcal{F}_X$$

for all $A \in \mathcal{F}_Y$.

Definition F.1 (Subspace σ-algebra). Let (X, Σ) be a Borel measurable space and let $Y \subseteq X$. The subspace (or trace) σ-algebra on Y is defined as

$$\Sigma_Y = \{Y \cap A \mid A \in \Sigma\}.$$

Then (Y, Σ_Y) is a Borel measurable space.

Definition F.2 (σ-algebra generated by a function). Let $f \colon X \to Y$ be a function and let (Y, Σ_Y) be a measurable space. The σ-algebra generated by f is defined by:

$$f^{-1}(\Sigma_Y) := \{f^{-1}(A) \mid A \in \Sigma_Y\}.$$

Then $(X, f^{-1}(\Sigma_Y))$ is a measurable space.

Lemma F.3 (Bijection induces isomorphism). *Let $f : X \to Y$ be a bijection and let (Y, Σ_Y) be a measurable space. Equip X with the σ-algebra generated by f:*

$$\Sigma_X := f^{-1}(\Sigma_Y) = \{f^{-1}(A) \mid A \in \Sigma_Y\}.$$

Then,

(1) (X, Σ_X) is a measurable space, and
(2) f induces an isomorphism of measurable spaces:

$$(X, \Sigma_X) \cong (Y, \Sigma_Y).$$

Proof. We first verify that i) (X, Σ_X) is a measurable space, then show that ii) f induces an isomorphism of measurable spaces $(X, \Sigma_X) \cong (Y, \Sigma_Y)$.

Proof of i) We show that $\Sigma_X = f^{-1}(\Sigma_Y)$ is a σ-algebra on X.

- **(Contains X):** $X = f^{-1}(Y) \in \Sigma_X$ since $Y \in \Sigma_Y$.
- **(Complements):** If $f^{-1}(A) \in \Sigma_X$, then $X \setminus f^{-1}(A) = f^{-1}(Y \setminus A) \in \Sigma_X$ since $Y \setminus A \in \Sigma_Y$.
- **(Countable unions):** If $f^{-1}(A_i) \in \Sigma_X$ for $i \in \mathbb{N}$, then $\bigcup_i f^{-1}(A_i) = f^{-1}(\bigcup_i A_i) \in \Sigma_X$ since $\bigcup_i A_i \in \Sigma_Y$.

Hence (X, Σ_X) is a measurable space.

Proof of ii) Two measurable spaces (X, Σ_X) and (Y, Σ_Y) are isomorphic iff there exists a bijection $f : X \to Y$ such that f and f^{-1} are measurable.

We first show that f is measurable. Measurability of f means that $f^{-1}(A) \in \Sigma_X$ for every $A \in \Sigma_Y$. Let $A \in \Sigma_Y$ be arbitrary. By definition of Σ_X,

$$f^{-1}(A) \in f^{-1}(\Sigma_Y) = \Sigma_X.$$

It follows that f is measurable.

Next we show that f^{-1} is measurable. Measurability of f^{-1} means that $(f^{-1})^{-1}(B) \in \Sigma_Y$ for every $B \in \Sigma_X$. Let $B \in \Sigma_X$ be arbitrary. By definition of Σ_X, there exists some $A \in \Sigma_Y$ such that $B = f^{-1}(A)$.

Observe that:

$$\begin{aligned}
(f^{-1})^{-1}(B) &= f(B) && \text{(preimage of } f^{-1} \text{ is } f\text{)} \\
&= f(f^{-1}(A)) && \text{(substituting } B = f^{-1}(A)\text{)} \\
&= A && (f \text{ is a bijection})
\end{aligned}$$

Since $A \in \Sigma_Y$, it follows that f^{-1} is measurable.

Therefore f is a measurable bijection with measurable inverse, so it is an isomorphism of measurable spaces. □

Lemma F.4 (Minimality of generated σ-algebras). *Let X be a set and let $\mathcal{G} \subseteq \mathcal{P}(X)$. Suppose $\mathcal{C}$ is a σ-algebra on X such that $\mathcal{G} \subseteq \mathcal{C}$. Then*

$$\sigma(\mathcal{G}) \subseteq \mathcal{C}.$$

Proof. Recall that $\sigma(\mathcal{G})$ is defined as the smallest σ-algebra on X containing $\mathcal{G}$, given concretely by the intersection of all σ-algebras on X containing $\mathcal{G}$:

$$\sigma(\mathcal{G}) = \bigcap_{\substack{\mathcal{F} \text{ is a } \sigma\text{-algebra on } X \\ \mathcal{G} \subseteq \mathcal{F}}} \mathcal{F}.$$

Since $\mathcal{C}$ is a σ-algebra on X containing $\mathcal{G}$, it appears in this intersection. Therefore,

$$\sigma(\mathcal{G}) \subseteq \mathcal{C}.$$

□

Definition F.5 (Markov kernels). Given measurable spaces (X, Σ_X) and (Y, Σ_Y), a Markov kernel with source X and target Y is a function $P : X \times \Sigma_Y \to [0, 1]$ (or equivalently, $X \to \Sigma_Y \to [0, 1]$) that maps each source state $x \in X$ to a distribution over target states $P(x, -) : \Sigma_Y \to [0, 1]$.

We call P a *Markov kernel* if it satisfies two conditions:

(1) *Probability measure:* For each source state $x \in X$, the map $A \mapsto P(x, A)$ must be a probability measure.
(2) *Measurable function:* For each event $A \in \Sigma_Y$ in the target space, the map $x \mapsto P(x, A)$ must be a measurable function.

Definition F.6 (Projection with error). Let $(X + \bot, \Sigma_{X+\bot})$ and $(Y + \bot, \Sigma_{Y+\bot})$ Let (X, Σ_X) and (Y, Σ_Y) be measurable spaces, with (X, Σ_X) and (Y, Σ_Y) measurablespaces as well. Let $\mu \in \mathcal{D}((X \times Y) + \bot)$. We define the first and second projection measures as : $\pi_1(\mu)(A) := \mu(A \times Y)$. Equip a subset $C \subseteq X \times Y$ with the trace

σ-algebra inherited from the product, and equip $X + \bot$, $Y + \bot$, and $C + \bot$ with their error-extension σ-algebras. Define

$$p_1 : C + \bot \longrightarrow X + \bot, \qquad p_2 : C + \bot \longrightarrow Y + \bot$$

by

$$p_i(\bot) = \bot, \qquad p_1(x, y) = x, \qquad p_2(x, y) = y.$$

Both maps are measurable. For example, for $A \in \Sigma_X$,

$$p_1^{-1}(A) = C \cap (A \times Y), \qquad p_1^{-1}(A \cup \{\bot\}) = (C \cap (A \times Y)) \cup \{\bot\},$$

and similarly for p_2. For $\mu \in \mathcal{D}(C + \bot)$, define its projections as the pushforwards

$$\pi_i(\mu) := (p_i)_* \mu, \qquad \pi_i(\mu)(E) = \mu(p_i^{-1}(E))$$

for every measurable $A \in \Sigma_X$, and $\pi_1(\mu)(\bot) := \mu(\bot)$ $\pi_2(\mu)(B) := \mu(X \times B)$ for every measurable $B \in \Sigma_Y$, and $\pi_2(\mu)(\bot) := \mu(\bot)$ Then, $\pi_1(\mu)$ is a measure on $(X + \bot, \Sigma_{X+\bot})$ and $\pi_2(\mu)$ is a measure on $(Y + \bot, \Sigma_{Y+\bot})$event E in the corresponding component space. Thus $\pi_1(\mu) \in \mathcal{D}(X + \bot)$ and $\pi_2(\mu) \in \mathcal{D}(Y + \bot)$. In particular, projection preserves the mass at $\bot$.

F.2 Measure Space on Expressions

To make the probabilistic semantics precise, we construct an appropriate measure space structure on the set of expressions by factoring out real numbers using the notion of what we call *expression skeletons*.

Definition F.7 (Expression Skeleton). An expression skeleton s is a syntactic expression obtained from a program by replacing each occurrence of a float constant with a hole of the form $\square$. The set of skeletons is generated by the grammar:

$$\begin{aligned} s ::= {} & \square \mid x \mid \text{true} \mid \text{false} \mid k_{\#n} \mid \textbf{diverge} \mid () \\ & \mid \textbf{let}\ x = s_1\ \textbf{in}\ s_2 \mid \textbf{if}\ s_1\ \textbf{then}\ s_2\ \textbf{else}\ s_3 \\ & \mid s_1 < s_2 \mid s_1 \le s_2 \mid s_1 <_{\#n} s_2 \mid s_1 \le_{\#n} s_2 \\ & \mid \textbf{uniform}(s_1, s_2) \mid \textbf{gaussian}(s_1, s_2) \mid \textbf{exponential}(s) \mid \textbf{beta}(s_1, s_2) \mid \textbf{discrete}(p_0, \ldots, p_{n-1}) \\ & \mid s_1 \ \&\& \ s_2 \mid s_1 \ || \ s_2 \mid \text{not } s \mid \textbf{observe}(s) \mid s_1; s_2 \\ & \mid (s_1, s_2) \mid \textbf{fst}\ s \mid \textbf{snd}\ s \\ & \mid \textbf{fun}\ x \to s \mid s_1\ s_2 \mid \textbf{fix}\ f\ x := s \\ & \mid \texttt{nil} \mid s_1 :: s_2 \\ & \mid (\textbf{match}\ s\ \textbf{with} \mid \texttt{nil} \to s_{\text{nil}} \mid h :: t \to s_{\text{cons}}\ \textbf{end}) \end{aligned}$$

We denote by $Skel$ the set of all expression skeletons, and by $holes(s)$ the number of distinct holes in skeleton s.

Example F.8. The expression $\textbf{uniform}(0.0, 1.0) < 0.5$ has the skeleton $\textbf{uniform}(\square, \square) < \square$ with three holes.

Definition F.9 (Filling a skeleton). Let $s \in Skel$ and let $\vec{r} = (r_1, \ldots, r_{holes(s)}) \in \mathbb{R}^{holes(s)}$. Define the expression $s[\vec{r}] \in Expr$ to be the result of replacing each hole $\square_i$ in s with the corresponding real constant r_i, for $1 \le i \le holes(s)$.

To recover a *typed* notion of skeletons, we extend the typing judgment to skeletons by adding a distinguished typing rule for holes.

Definition F.10 (Well-typed expression skeleton). Define a typing judgment for skeletons, written $\Gamma \vdash s : \tau$, with the understanding that all typing rules from expressions are preserved for skeletons, and that holes are typed as $\textbf{float}[B; V]$ according to the rule below:

$$\frac{}{\square : \textbf{float}[B; V]}$$

We denote by $Skel_\tau$ the set of all well-typed expression skeletons, where:

$$Skel_\tau := \{s \in Skel \mid s : \tau\}.$$

(ii) Closure under complements: Let $A \in \mathcal{F}_{Expr_\tau}$. We show that $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$. By Definition F.23, it suffices to show that $\forall s \in Skel_\tau$, $(Expr_\tau \setminus A) \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.

Fix $s \in Skel_\tau$. Observe that:

$$\begin{aligned} & (Expr_\tau \setminus A) \cap Expr_{s;\tau} \\ &= (Expr_\tau \cap Expr_{s;\tau}) \setminus (A \cap Expr_{s;\tau}) && (\cap \text{ distributes over } \setminus) \\ &= Expr_{s;\tau} \setminus (A \cap Expr_{s;\tau}). && (\text{since } Expr_{s;\tau} \subseteq Expr_\tau, \text{ so } Expr_\tau \cap Expr_{s;\tau} = Expr_{s;\tau}) \end{aligned}$$

Since $A \in \mathcal{F}_{Expr_\tau}$ by assumption, by Definition F.23, we have $A \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$, and because $\mathcal{F}_{Expr_{s;\tau}}$ is a σ-algebra, it is closed under complements relative to $Expr_{s;\tau}$. Hence

$$Expr_{s;\tau} \setminus (A \cap Expr_{s;\tau}) \in \mathcal{F}_{Expr_{s;\tau}}.$$

Therefore $(Expr_\tau \setminus A) \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for all $s \in Skel_\tau$. Therefore, $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$.

(iii) Closure under countable unions: Let $(A_n)_{n\in\mathbb{N}}$ be a sequence with $A_n \in \mathcal{F}_{Expr_\tau}$ for all n. We show that $\bigcup_{n\in\mathbb{N}} A_n \in \mathcal{F}_{Expr_\tau}$. By Definition F.23, it suffices to show that $\forall s \in Skel_\tau$, $(\bigcup_{n\in\mathbb{N}} A_n) \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.

Fix $s \in Skel_\tau$. Using distributivity of $\cap$ over $\cup$, we have

$$\left(\bigcup_{n\in\mathbb{N}} A_n\right) \cap Expr_{s;\tau} = \bigcup_{n\in\mathbb{N}} \left(A_n \cap Expr_{s;\tau}\right).$$

Since each $A_n \in \mathcal{F}_{Expr_\tau}$ by assumption, by Definition F.23, we have $A_n \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for all n. Because $\mathcal{F}_{Expr_{s;\tau}}$ is a σ-algebra, it is closed under countable unions, hence

$$\bigcup_{n\in\mathbb{N}} \left(A_n \cap Expr_{s;\tau}\right) \in \mathcal{F}_{Expr_{s;\tau}}.$$

Therefore $(\bigcup_{n\in\mathbb{N}} A_n) \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for all s, and thus $\bigcup_{n\in\mathbb{N}} A_n \in \mathcal{F}_{Expr_\tau}$.

Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra on $Expr_\tau$, $(Expr_\tau, \mathcal{F}_{Expr_\tau})$ is a measurable space. □

Definition F.25 (Extension with error element). Let $(Expr_\tau, \mathcal{F}_{Expr_\tau})$ be the measurable space of well-typed expressions of type τ. Define For any measurable space (X, Σ_X) and a fresh element $\bot \notin X$, define

$$X + \bot := X \cup \{\bot\},$$

equipped with the σ-algebra

$$\Sigma_{X+\bot} := \{A,\ A \cup \{\bot\} \mid A \in \Sigma_X\}.$$

In particular, taking $X = Expr_\tau$ gives $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$.

Lemma F.26. *$(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ is a measurable space.*

Proof. We verify that $\mathcal{F}_{Expr_\tau+\bot}$ is indeed a σ-algebra on $Expr_\tau + \bot$.

By construction, every element of $\mathcal{F}_{Expr_\tau+\bot}$ is of the form A or $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$.

(i) Contains $Expr_\tau + \bot$: Take $A = Expr_\tau \in \mathcal{F}_{Expr_\tau}$. Then,

$$A \cup \{\bot\} = Expr_\tau \cup \{\bot\} = Expr_\tau + \bot \in \mathcal{F}_{Expr_\tau+\bot}.$$

(ii) Closure under complements: Let $C \in \mathcal{F}_{Expr_\tau+\bot}$. There are two cases:

- If $C = A$: Then, $(Expr_\tau + \bot) \setminus A = (Expr_\tau \setminus A) \cup \{\bot\}$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$, so $(Expr_\tau \setminus A) \cup \{\bot\}$ is of the form $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$ and hence lies in $\mathcal{F}_{Expr_\tau+\bot}$.
- If $C = A \cup \{\bot\}$: Then, $(Expr_\tau + \bot) \setminus (A \cup \{\bot\}) = Expr_\tau \setminus A$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$, so $Expr_\tau \setminus A$ is of the form A for some $A \in \mathcal{F}_{Expr_\tau}$ and hence lies in $\mathcal{F}_{Expr_\tau+\bot}$.

In both cases the complement lies in $\mathcal{F}_{Expr_\tau+\bot}$.

(iii) Closure under countable unions: Let $(C_n)_{n\in\mathbb{N}}$ be a sequence with $C_n \in \mathcal{F}_{Expr_\tau+\bot}$ for all n. For each n, define $A_n := C_n \cap Expr_\tau \in \mathcal{F}_{Expr_\tau}$, where the latter holds since C_n is of the form A or $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$, and in either case $C_n \cap Expr_\tau = A$.

Let $A := \bigcup_{n\in\mathbb{N}} A_n$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $A \in \mathcal{F}_{Expr_\tau}$. Now observe:

$$\bigcup_{n\in\mathbb{N}} C_n = A \cup \left(\bigcup_{n\in\mathbb{N}} (C_n \setminus Expr_\tau)\right).$$

The second factor is either $\emptyset$ (if every $C_n = A_n$) or $\{\bot\}$ (if some $C_n = A_n \cup \{\bot\}$). In the first sub-case, $\bigcup_n C_n = A$ is of the form A for some $A \in \mathcal{F}_{Expr_\tau}$; in the second, $\bigcup_n C_n = A \cup \{\bot\}$ is of the form $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$. In both cases $\bigcup_n C_n \in \mathcal{F}_{Expr_\tau+\bot}$. □

Finally, we can equip this measurable space $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ with a subprobability measure such that $\mu(Expr_\tau + \bot) \le 1$.

We give some basic lemmas:

Lemma F.27. *Let $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ be the Borel measurable space of well-typed expressions with error from Definition F.25. Let $Vals_\tau \subset Expr_\tau$ be the set of all well-typed value expressions. Let $N_\tau \subset Expr_\tau$, defined as $N = Expr_\tau \setminus Vals_\tau$, be the set of all well-typed non-value expressions. Let $\{\bot\}$ be the singleton set containing the error element $\bot$. The following are true:*

(1) The set $Vals_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.
(2) The set $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.
(3) The set $N_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.

Proof. We prove the measurability of $Vals_\tau$, $\{\bot\}$, and N_τ in separate parts. For each, we show that it belongs to the σ-algebra $\mathcal{F}_{Expr_\tau+\bot}$ on the extended expression space $Expr_\tau + \bot$.

(1) **(Measurability of $Vals_\tau$):** Every expression skeleton $s \in Skel_\tau$ is either a value skeleton or a non-value skeleton, depending on whether the syntactic form of s is a value. An expression $e \in Expr_\tau$ is a value iff its skeleton is a value skeleton.
To show $Vals_\tau \in \mathcal{F}_{Expr_\tau}$, by Definition F.23, it suffices to show that $\forall s \in Skel_\tau, Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.
Fix $s \in Skel_\tau$:
- If s is a value skeleton, then every expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.
- If s is a non-value skeleton, then no expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = \emptyset \in \mathcal{F}_{Expr_{s;\tau}}$.

Thus, $Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for any $s \in Skel_\tau$, so $Vals_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

(2) **(Measurability of $\{\bot\}$):** Taking $A = \emptyset$ in Definition F.25 gives $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.

(3) **(Measurability of N_τ):** $N_\tau = Expr_\tau \setminus Vals_\tau$ is the complement of $Vals_\tau$ in $Expr_\tau$. Since $Vals_\tau \in \mathcal{F}_{Expr_\tau}$ and $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $N_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

□

Note that in general, given (X, Σ_X) and (Y, Σ_Y) to be measurable spaces, $((X \times Y) + \bot)$ forms a measurable space, since the product $(X \times Y)$ is measurable and for any measurable space (Z, Σ_Z), the space $(Z + \bot, \Sigma_{Z+\bot})$ with $\Sigma_{Z+\bot}$ generated by $\Sigma_Z \cup \bot$ is measurable.

F.3 Distribution Monad Strict Sub-Markov Kleisli Composition with Error

Let X be a set equipped with a suitable measurable space so that $\mathcal{D}(X + \bot)$ denotes the set of probability measures over $X + \bot$. We will make heavy use of the following generic monad construction, for which one easily verifies the monad laws: measurable space and let $\mathcal{D}(X + \bot)$ denote the subprobability measures on its error extension.

Definition F.28 (Strict sub-Markov Kleisli bind). We define

$$\begin{aligned}&\text{return}\colon X \to \mathcal{D}(X+\bot), \quad \text{return}(x) = \delta_x\\ &\text{bind}\colon \mathcal{D}(X+\bot) \to (X \to \mathcal{D}(Y+\bot)) \to \mathcal{D}(Y+\bot),\\ &\text{bind}(\mu)(f)(A) = \int_{X+\bot} f_\bot(x)(A)\cdot \mu(dx)\,,\end{aligned}$$

where δ_x denotes the Dirac measure for x, and f is a Markov kernel . $f : X \to \mathcal{D}(Y+\bot)$ is a sub-Markov kernel and $f_\bot$ is its canonical extension from Definition F.29. Thus the continuation is specified only on ordinary inputs and $\delta_\bot \gg\!= f = \delta_\bot$; bind cannot recover from an error.

Definition F.29 ($\bot$-extension). Define the $\bot$-extension of $g : X \to \mathcal{D}(Y+\bot)$ to be $g_\bot\colon (X+\bot) \to \mathcal{D}(Y+\bot)$, obtained from g by:

$$g_\bot(e) = \begin{cases}\delta_\bot & \text{if } e = \bot\\ g(e) & \text{otherwise}\,.\end{cases}$$

Lemma F.30 ($\bot$-extension is a sub-Markov kernel). *Given a Markov sub-Markov kernel $f : X \to \mathcal{D}(Y+\bot)$, its $\bot$ extension $f_\bot : (X+\bot) \to \mathcal{D}(Y+\bot)$ is also a Markov kernel. sub-Markov kernel. If f is Markov, its extension is Markov.*

Proof. We verify the two conditions for $f_\bot$ to be a Markov sub-Markov kernel.

(1) Probability Subprobability measure: We show for each $e \in X+\bot$, the map

$$A \mapsto f_\bot(e)(A)$$

is a probability measure.

Fix $e \in X+\bot$.

- If $e = \bot$, then $f_\bot(\bot) = \delta_\bot$, the Dirac measure at $\bot$, which is a probability measure on $Y+\bot$.
- If $e \in X$, then $f_\bot(e) = f(e)$, and since f is a Markov kernel by assumption, $f(e)$ is a probability measure on $Y+\bot$a subprobability measure by assumption.

Thus , for all $e \in X+\bot$, the map $A \mapsto f_\bot(e)(A)$ is a probability each map $A \mapsto f_\bot(e)(A)$ is a subprobability measure.

(2) Measurable function: We show for each $A \in \mathcal{F}_{Y+\bot}$, the map $e \mapsto f_\bot(e)(A)$ is a measurable function.

Fix $A \in \mathcal{F}_{Y+\bot}$. Define

$$g\colon (X+\bot) \to [0,1], \qquad g(e) := f_\bot(e)(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_{X+\bot}$ for every $B \in \mathcal{B}([0,1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}((a,\infty)\cap[0,1]) \in \mathcal{F}_{X+\bot}$$

because the family of sets $\{(a,\infty)\cap[0,1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0,1])$.

Because $g(e) \in [0,1]$ for all $e \in X+\bot$ by definition, we have for every $a \in \mathbb{R}$,

$$\begin{aligned}g^{-1}\big((a,\infty)\cap[0,1]\big) &= \{e \in X+\bot \mid g(e) \in (a,\infty)\cap[0,1]\}\\ &= \{e \in X+\bot \mid (g(e)\in(a,\infty)) \wedge (g(e)\in[0,1])\}\\ &= \{e \in X+\bot \mid (g(e) > a) \wedge (g(e)\in[0,1])\}\\ &= \{e \in X+\bot \mid g(e) > a\},\end{aligned}$$

where the last step uses that $g(e) \in [0,1]$ always holds. Hence it suffices to show that $\{e \in X+\bot \mid g(e) > a\} \in \mathcal{F}_{X+\bot}$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{e \in X+\bot \mid g(e) > a\} = \emptyset \in \mathcal{F}_{X+\bot}$.
- If $a < 0$: $\{e \in X+\bot \mid g(e) > a\} = X+\bot \in \mathcal{F}_{X+\bot}$.
- If $0 \leq a < 1$: $\{e \in X+\bot \mid g(e) > a\}$.

The only nontrivial case is the last. It splits into two sets:

$$\begin{aligned}\{e \in X + \bot \mid g(e) > a\} &= \{e \in X + \bot \mid f_\bot(e)(A) > a\} \\ &= \{e \in X \mid f_\bot(e)(A) > a\} \cup \{\bot \mid f_\bot(e)(A) > a\} \\ &= \{e \in X \mid f(e)(A) > a\} \cup \{\bot \mid \delta_\bot(A) > a\}\end{aligned}$$

The first set is measurable in X (and hence $X+\bot$) since f is a Markov kernelby assumptionsub-Markov kernel. The second set is either $\emptyset \in \mathcal{F}_{X+\bot}$ or $\{\bot\} \in \mathcal{F}_{X+\bot}$. Since $\mathcal{F}_{X+\bot}$ is closed under countable unions, then $g^{-1}(B) \in \mathcal{F}_{X+\bot}$ for every $B \in \mathcal{B}([0,1])$. Thus , for all $A \in \mathcal{F}_{Y+\bot}$, the map $e \mapsto f_\bot(e)(A)$ is a measurablefunction$\emptyset$ or $\{\bot\}$. Thus the union is measurable.

Since both conditions hold, $f_\bot$ is a Markov kernel. sub-Markov kernel. If f is Markov, both cases have total mass one, so $f_\bot$ is Markov. □

Lemma F.31 (Dirac return is a Markov kernel). *For any measurable space $(X, \mathcal{F}_X)$, the map*

$$\mathsf{return} : X \to \mathcal{D}(X + \bot), \qquad x \mapsto \delta_x.$$

is a Markov kernel.

Proof. We verify the two conditions for return to be a Markov kernel.

(1) Probability measure: We show for each $x \in X$, the map

$$A \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a probability measure.

Fix $x \in X$. δ_x is the Dirac measure at x, hence a probability measure on $X + \bot$. Therefore, for every $x \in X$, the map $A \mapsto \delta_x(A)$ is a probability measure.

(2) Measurable function: We show that for each $A \in \mathcal{F}_{X+\bot}$, the map

$$x \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a measurable function.

Fix $A \in \mathcal{F}_{X+\bot}$. Define

$$g : X \to [0,1], \qquad g(x) := \delta_x(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0,1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}\big((a,\infty) \cap [0,1]\big) \in \mathcal{F}_X,$$

because the family $\{(a,\infty) \cap [0,1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0,1])$.

Because $g(e) \in [0,1]$ for all $e \in X + \bot$, we have for every $a \in \mathbb{R}$,

$$\begin{aligned}g^{-1}\big((a,\infty) \cap [0,1]\big) &= \{e \in X \mid g(e) \in (a,\infty) \cap [0,1]\} \\ &= \{e \in X \mid (g(e) \in (a,\infty)) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid (g(e) > a) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid g(e) > a\},\end{aligned}$$

where the last step uses that $g(e) \in [0,1]$ always holds. Hence it suffices to show that $\{e \in X \mid g(e) > a\} \in \mathcal{F}_X$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{x \in X \mid g(x) > a\} = \emptyset \in \mathcal{F}_X$.
- If $a < 0$: $\{x \in X \mid g(x) > a\} = X \in \mathcal{F}_X$.
- If $0 \leq a < 1$: $\{x \in X \mid g(x) > a\}$.

The only nontrivial case is the last. Using $g(x) = \delta_x(A)$ and the fact that $\delta_x(A) \in \{0, 1\}$ for all $x \in X$, we have for $0 \leq a < 1$:

$$\begin{aligned}\{x \in X \mid g(x) > a\} &= \{x \in X \mid \delta_x(A) > a\} \\ &= \{x \in X \mid \delta_x(A) = 1\} \\ &= \{x \in X \mid x \in A\} \\ &= A \cap X.\end{aligned}$$

Since $A \in \mathcal{F}_{X+\perp}$ and $\mathcal{F}_X = \{X \cap A \mid A \in \mathcal{F}_{X+\perp}\}$ is the subspace σ-algebra on $X \subseteq X + \perp$, it follows that $A \cap X \in \mathcal{F}_X$.

Therefore $\{x \in X \mid g(x) > a\} \in \mathcal{F}_X$ for all $a \in \mathbb{R}$, so $g^{-1}\big((a, \infty) \cap [0, 1]\big) \in \mathcal{F}_X$ for all a. Hence $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0, 1])$, and g is measurable. □

Lemma F.32 (Zero kernel is a sub-Markov kernel). *Let $(X, \mathcal{F}_X)$ and $(Y, \mathcal{F}_Y)$ be measurable spaces. The constant-zero map*

$$f : X \to \mathcal{D}(Y), \qquad f(x)(A) := 0$$

is a Markov sub-Markov kernel.

Proof. We verify the two kernel conditions.

(1) Probability Subprobability measure. We show for each $x \in X$, the map

$$A \mapsto f(x)(A)$$

is a probability measure.

Fix any $x \in X$. Then $f(x)$ is the zero measure: it satisfies σ-additivity since

$$f(x)\left(\bigcup_i A_i\right) = 0 = \sum_i 0 = \sum_i f(x)(A_i)$$

for every pairwise disjoint family $(A_i)_i$, and total mass $f(x)(Y) = 0 \leq 1$.

(2) Measurable function. For any $A \in \mathcal{F}_Y$, the map $x \mapsto f(x)(A) = 0$ is the constant-zero function on X, hence measurable.

Thus both conditions hold, so f is a Markov sub-Markov kernel. □

Lemma F.33 (Composition of sub-Markov kernels). *Let (X, Σ_X), (Y, Σ_Y), and (Z, Σ_Z) be measurable spaces. If*

$$P : X \to \mathcal{D}(Y) \quad \text{and} \quad Q : Y \to \mathcal{D}(Z)$$

are Markov sub-Markov kernels, then their composition

$$(P \gg\!= Q)(A) \;:=\; \int_{x \in supp(P)} Q(x)(A)\, P(dx)$$

defines a Markov sub-Markov kernel

$$P \gg\!= Q : X \to \mathcal{D}(Z).$$

Proof. See [30, Lemma 1.38]. □

We claim proving $C \in \mathcal{F}_{Expr_{\tau_1}}$ is sufficient because by Definition F.1, this gives $\mathrm{Sub}_{e,x}^{-1}(A) = Val_{\tau_1} \cap C \in \mathcal{F}_{Val_{\tau_1}}$. To show $C \in \mathcal{F}_{Expr_{\tau_1}}$, by Definition F.23, it suffices to show that $\forall s \in Skel_{\tau_1}, C \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}}$.

Fix $s \in Skel_{\tau_1}$. We must prove:

$$\begin{aligned}
& C \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}} \\
\iff & \{u \in Expr_{\tau_1} \mid e[u/x] \in A\} \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}} && \text{(definition of } C\text{)} \\
\iff & \{u \in Expr_{s;\tau_1} \mid e[u/x] \in A\} \in \mathcal{F}_{Expr_{s;\tau_1}} && (Expr_{s;\tau_1} \subseteq Expr_{\tau_1}) \\
\iff & \{s[\vec{r}] \in Expr_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \in \mathcal{F}_{Expr_{s;\tau_1}} && \text{(rewrite } u\text{)} \\
\iff & \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \in \mathcal{F}_{s;\tau_1} && \text{(Lemma F.21)}
\end{aligned}$$

Define

$$R_s := \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\}.$$

Let $t \in Skel_{\tau_2}$ be the skeleton obtained from e by replacing each occurrence of x with s. Define a constant extension map:

$$\psi_s : A_{s;\tau_1} \to A_{t;\tau_2}, \qquad \psi_s(\vec{r}) = \phi(\vec{r}, \vec{c_1}, ..., \vec{c_k})$$

where $\vec{c_1}, ..., \vec{c_k} \in \mathbb{R}$ are the float constants appearing in e (other than at occurrences of x), and ϕ permutes/interleaves the blocks into the order dictated by a left-to-right traversal of the holes of t.

More precisely, since t is obtained from e by replacing each free occurrence of x with s, the holes of t consist exactly of:

- the holes of s, repeated once per occurrence of x in e, and
- the holes coming from the float constants of e itself.

By Lemma F.35, ψ_s is measurable.

By construction, for all $\vec{r} \in A_{s;\tau_1}$:

$$e[s[\vec{r}]/x] = t[\psi_s(\vec{r})].$$

Define the measurable set of hole-assignments corresponding to A at skeleton t:

$$\begin{aligned}
A_t :=& \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A\} \\
:=& \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A \cap Expr_{t;\tau_2}\} && (t[\vec{u}] \text{ ranges over } Expr_{t;\tau_2})
\end{aligned}$$

By Definition F.23, $A \cap Expr_{t;\tau_2} \in \mathcal{F}_{Expr_{t;\tau_2}}$, and by Lemma F.3, $A_t \in \mathcal{F}_{t;\tau_2}$. We compute:

$$\begin{aligned}
R_s &= \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \\
&= \{\vec{r} \in A_{s;\tau_1} \mid t[\psi_s(\vec{r})] \in A\} \\
&= \{\vec{r} \in A_{s;\tau_1} \mid \psi_s(\vec{r}) \in A_t\} \\
&= \psi_s^{-1}(A_t).
\end{aligned}$$

Since ψ_s is measurable and $A_t \in \mathcal{F}_{t;\tau_2}$, we have $R_s = \psi_s^{-1}(A_t) \in \mathcal{F}_{s;\tau_1}$.

Since $R_s \in \mathcal{F}_{s;\tau_1}$ for every $s \in Skel_{\tau_1}$, we get $C \in \mathcal{F}_{Expr_{\tau_1}}$, and therefore

$$\mathrm{Sub}_{e,x}^{-1}(A) = C \cap Val_{\tau_1} \in \mathcal{F}_{Val_{\tau_1}}.$$

Hence $\mathrm{Sub}_{e,x}$ is measurable. □

Conjecture F.38 (Strict bind continuations). *For every occurrence of a monadic bind $\mu \gg\!= f$ given $\mu \gg\!= f$ in the small-step operational semantics in Tables 1 and 2, $f : (Expr_{\tau_1} + \bot) \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a Markov kernel. rules, $f : Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is an ordinary-input sub-Markov kernel, conditional on the stated measurability premises. Its value at $\bot$ is supplied only by the canonical strict extension.*

Conjecture F.39 (Small-step semantics is a sub-Markov kernel). *For Conditional on the sampler and case-partition measurability premises, for each type τ , the small-step operational semantics $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ defines a Markov kernel. $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a sub-Markov kernel.*

$\mathbf{e} : \boldsymbol{\tau}$	**constraints**	$[\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]$
$\mathbf{diverge} : \tau$		$\lambda A.0$
$v : \tau$		$\delta_{(v:\tau, \mathcal{D}[\![v:\tau]\!]:\mathcal{D}[\![\tau]\!])}$
$\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2 : \tau_2$	$e_1 = v$ e_1 no val.	$\delta_{(e_2[v/x]:\tau_2, \mathcal{D}[\![e_2:\tau_2]\!][\mathcal{D}[\![v:\tau_1]\!]/x]:\mathcal{D}[\![\tau_2]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{let}\ x=g\ \mathbf{in}\ e_2:\tau_2:\tau_2, \mathbf{let}\ x=g'\ \mathbf{in}\ \mathcal{D}[\![e_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]:\mathcal{D}[\![\tau_2]\!])}$
$\mathbf{if}\ e_1 : \mathbf{bool}\ \mathbf{then}\ e_2 : \tau\ \mathbf{else}\ e_3 : \tau : \tau$	$e_1 = \text{true}$ $e_1 = \text{false}$ o.w.	$\delta_{(e_2:\tau, \mathcal{D}[\![e_2:\tau]\!]:\mathcal{D}[\![\tau]\!])}$ $\delta_{(e_3:\tau, \mathcal{D}[\![e_3:\tau]\!]:\mathcal{D}[\![\tau]\!])}$ $[\![e_1 : \mathbf{bool}, \mathcal{D}[\![e_1 : \mathbf{bool}]\!] : \mathcal{D}[\![\mathbf{bool}]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{if}\ g\ \mathbf{then}\ e_2:\tau\ \mathbf{else}\ e_3:\tau:\tau,\ \mathbf{if}\ g'\ \mathbf{then}\ \mathcal{D}[\![e_2:\tau]\!]:\mathcal{D}[\![\tau]\!]\ \mathbf{else}\ \mathcal{D}[\![e_3:\tau]\!]:\mathcal{D}[\![\tau]\!]:\mathcal{D}[\![\tau]\!])}$
$\mathbf{observe}\ (e : \mathbf{bool}) : \mathbf{unit}$	$e = \text{true}$ $e = \text{false}$ o.w.	$\delta_{(\mathbf{unit}, \mathbf{unit})}$ $\delta_{\bot}$ $[\![e : \mathbf{bool}, \mathcal{D}[\![e]\!] : \mathbf{bool}]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{observe}\ (g:\mathbf{bool}):\mathbf{unit}, \mathbf{observe}\ (g':\mathbf{bool}):\mathbf{unit})}$
$e_1 : \tau_1 < e_2 : \tau_2 : \mathbf{bool}$ for $\tau_1 = \mathbf{float}[B; V_1], \tau_2 = \mathbf{float}[B; V_2]$ $B = \top$	$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{(\text{true}:\mathbf{bool}, \text{true}:\mathbf{bool})}$ $\delta_{(\text{false}:\mathbf{bool}, \text{false}:\mathbf{bool})}$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2 : \tau_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(v_1:\tau_1 < g:\mathbf{bool}, \mathcal{D}[\![v_1:\tau_1]\!]:\mathcal{D}[\![\tau_1]\!] < g':\mathcal{D}[\![\mathbf{bool}]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(g < e_2:\tau_2:\mathbf{bool}, g' < \mathcal{D}[\![e_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]:\mathcal{D}[\![\mathbf{bool}]\!])}$
$e_1 : \tau_1 < e_2 : \tau_2 : \mathbf{bool}$ for $\tau_1 = \mathbf{float}[B; V_1], \tau_2 = \mathbf{float}[B; V_2]$ $B \neq \top$	$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{(\text{true}:\mathbf{bool}, \text{true}:\mathbf{bool})}$ $\delta_{(\text{false}:\mathbf{bool}, \text{false}:\mathbf{bool})}$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2 : \tau_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(v_1:\tau_1 < g:\mathbf{bool}, \mathcal{D}[\![v_1:\tau_1]\!]:\mathcal{D}[\![\tau_1]\!] <_{\#(\lvert B\rvert+1)} g':\mathcal{D}[\![\mathbf{bool}]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(g < e_2:\tau_2:\mathbf{bool}, g' <_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]:\mathcal{D}[\![\mathbf{bool}]\!])}$
$\mathbf{uniform}(e_1 : \tau_1, e_2 : \tau_2) : \mathbf{float}[B; V]$ $B = \top$	$e_1 = v_1 \leq v_2 = e_2$ $e_1 = v_1 > v_2 = e_2$ $e_1 = v_1, e_2$ no val. e_1 no val.	$\text{Uniform}(v_1, v_2) \gg\!= \lambda v.\delta_{(v:\mathbf{float}[B;V], v:\mathbf{float}[B;V])}$ $[\![\mathbf{diverge} : \tau, \mathcal{D}[\![\mathbf{uniform}(v_1 : \tau_1, v_2 : \tau_2)]\!] : \mathcal{D}[\![\tau]\!]]\!]$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{uniform}(v_1:\tau_1, g:\tau_2):\tau, \mathbf{uniform}(\mathcal{D}[\![v_1]\!]:\mathcal{D}[\![\tau_1]\!], g':\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{uniform}(g:\tau_1, e_2:\tau_2):\tau, \mathbf{uniform}(g':\mathcal{D}[\![\tau_1]\!], \mathcal{D}[\![e_2]\!]:\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau]\!])}$
$\mathbf{uniform}(e_1 : \tau_1, e_2 : \tau_2) : \mathbf{float}[B; V]$ $B \neq \top$ $\tau_1 = \mathbf{float}[B_1 \neq \top, \{v_1, \ldots, v_n\}]$ $\tau_2 = \mathbf{float}[B_2 \neq \top, \{w_1, \ldots, w_m\}]$	$e_1 = c_1 \leq c_2 = e_2$ values $e_1 = v_1 > v_2 = e_2$ $e_1 = c_1, e_2$ no val. e_1 no val.	$\text{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{(v:\mathbf{float}[B;V], v_i:\mathbf{fin}(\lvert B\rvert+1))}$ where v_i is unique number s.t. $v \in \text{Intervals}(B)_{v_i}$ $[\![\mathbf{diverge} : \tau, \mathcal{D}[\![\mathbf{uniform}(v_1 : \tau_1, v_2 : \tau_2)]\!] : \mathcal{D}[\![\tau]\!]]\!]$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{uniform}(v_1:\tau_1, g:\tau_2):\tau, \mathsf{DC}_{B,B_1,B_2,V_1,V_2}(\mathcal{D}[\![v_1]\!]:\mathcal{D}[\![\tau_1]\!], g':\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{uniform}(g:\tau_1, e_2:\tau_2):\tau, \mathsf{DC}_{B,B_1,B_2,V_1,V_2}(g':\mathcal{D}[\![\tau_1]\!], \mathcal{D}[\![e_2]\!]:\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau]\!])}$

Table 3. Small-step coupling semantics. Notice that $\mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]$ is uniquely determined by $e : \boldsymbol{\tau}$ so that it suffices to indicate the latter in the left-hand column. Here, every function of the form $f(g, g') = \lambda(g, g').\delta_{\ldots}$ is extended to $f_\bot$ in the sense of Definition F.29.

F.5 Small-Step Coupling Semantics

Define $P_\tau = \{(e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]) \mid e : \tau\}$. Exploiting that $e : \tau$ is a value iff $\mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]$ is a value, we define : The small-step coupling $[\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!] \in \mathcal{D}(P_\tau + \bot)$ is defined recursively on the structure of $e : \tau$ by the rules in Tables 3 and 4.

Definition F.40 (Candidate exact-graph relation). Use $Expr_\tau^{\mathsf{U}}$ for the source fragment and $Expr_{\mathcal{D}[\![\tau]\!]}^{\mathsf{U}+}$ for its generated-target operational closure, as in the restricted operational semantics. Assuming these carriers have the measurable structures constructed above, define

$$P_\tau := \{(e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]) \mid e \in Expr_\tau^{\mathsf{U}} \text{ is closed}\} \subseteq Expr_\tau^{\mathsf{U}} \times Expr_{\mathcal{D}[\![\tau]\!]}^{\mathsf{U}+}$$

$\mathbf{e} : \boldsymbol{\tau}$	constraints	$[\![\boldsymbol{e} : \boldsymbol{\tau}, \mathcal{D}[\![\boldsymbol{e}]\!] : \mathcal{D}[\![\boldsymbol{\tau}]\!]]\!]$
$e_1 : \tau_1$ && $e_2 : \tau_2 : \mathbf{bool}$ $\tau_1 = \mathbf{bool}, \tau_2 = \mathbf{bool}$	$e_1 = b_1 = \text{true},\ e_2 = b_2 = \text{true}$ $e_1 = b_1,\ e_2 = b_2$ $e_1 = b_1,\ e_2 \neq b_2$ o.w.	$\delta_{(\text{true}:\mathbf{bool},\text{true}:\mathbf{bool})}$ $\delta_{(\text{false}:\mathbf{bool},\text{false}:\mathbf{bool})}$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2 : \tau_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(b_1:\tau_1 \&\& g:\mathbf{bool},\ b_1:\mathcal{D}[\![\tau_1]\!] \&\& g':\mathcal{D}[\![\mathbf{bool}]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(g \&\& e_2:\tau_2:\mathbf{bool},\ g' \&\& \mathcal{D}[\![e_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]:\mathcal{D}[\![\mathbf{bool}]\!])}$
$e_1 : \tau_1$ || $e_2 : \tau_2 : \mathbf{bool}$ $\tau_1 = \mathbf{bool}, \tau_2 = \mathbf{bool}$	$e_1 = b_1 = \text{false},\ e_2 = b_2 = \text{false}$ $e_1 = b_1,\ e_2 = b_2$ $e_1 = b_1,\ e_2 \neq b_2$ o.w.	$\delta_{(\text{false}:\mathbf{bool},\text{false}:\mathbf{bool})}$ $\delta_{(\text{true}:\mathbf{bool},\text{true}:\mathbf{bool})}$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2 : \tau_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(b_1:\tau_1 \,\|\|\, g:\mathbf{bool},\ b_1:\mathcal{D}[\![\tau_1]\!] \,\|\|\, g':\mathcal{D}[\![\mathbf{bool}]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(g \,\|\, e_2:\tau_2:\mathbf{bool},\ g' \,\|\, \mathcal{D}[\![e_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]:\mathcal{D}[\![\mathbf{bool}]\!])}$
not $e : \tau : \mathbf{bool}$ $\tau = \mathbf{bool}$	$e = \text{true}$ $e = \text{false}$ o.w.	$\delta_{(\text{false}:\mathbf{bool},\text{false}:\mathbf{bool})}$ $\delta_{(\text{true}:\mathbf{bool},\text{true}:\mathbf{bool})}$ $[\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\text{not } g:\mathbf{bool},\ \text{not } g':\mathcal{D}[\![\mathbf{bool}]\!])}$
$(e_1 : \tau_1, e_2 : \tau_2) : \tau_1 * \tau_2$	$e_1 = v_1,\ e_2 = v_2$ $e_1 = v_1,\ e_2 \neq v_2$ o.w.	$\delta_{((v_1,v_2):\tau_1*\tau_2,\ (\mathcal{D}[\![v_1]\!]:\mathcal{D}[\![\tau_1]\!], \mathcal{D}[\![v_2]\!]:\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau_1]\!]*\mathcal{D}[\![\tau_2]\!])}$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2 : \tau_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{((v_1:\tau_1, g:\tau_2):\tau_1*\tau_2,\ (\mathcal{D}[\![v_1]\!]:\mathcal{D}[\![\tau_1]\!], g':\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau_1]\!]*\mathcal{D}[\![\tau_2]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{((g, e_2:\tau_2):\tau_1*\tau_2,\ (g', \mathcal{D}[\![e_2:\tau_2]\!]):\mathcal{D}[\![\tau_1]\!]*\mathcal{D}[\![\tau_2]\!])}$
$\mathbf{fst}\ (e : \tau_1 * \tau_2) : \tau_1$	$e = (v_1, v_2)$ o.w.	$\delta_{(v_1:\tau_1,\ \mathcal{D}[\![v_1:\tau_1]\!]:\mathcal{D}[\![\tau_1]\!])}$ $[\![e : \tau_1 * \tau_2, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau_1 * \tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{fst}\ (g:\tau_1*\tau_2):\tau_1,\ \mathbf{fst}\ (g':\mathcal{D}[\![\tau_1*\tau_2]\!]):\mathcal{D}[\![\tau_1]\!])}$
$\mathbf{snd}\ (e : \tau_1 * \tau_2) : \tau_2$	$e = (v_1, v_2)$ o.w.	$\delta_{(v_2:\tau_2,\ \mathcal{D}[\![v_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!])}$ $[\![e : \tau_1 * \tau_2, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau_1 * \tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{snd}\ (g:\tau_1*\tau_2):\tau_2,\ \mathbf{snd}\ (g':\mathcal{D}[\![\tau_1*\tau_2]\!]):\mathcal{D}[\![\tau_2]\!])}$
$\mathbf{fun}\ x \to e : \tau_1 \to \tau_2$		$\delta_{(\mathbf{fun}\ x \to e:\tau_1\to\tau_2,\ \mathbf{fun}\ x \to \mathcal{D}[\![e]\!]:\mathcal{D}[\![\tau_1]\!]\to\mathcal{D}[\![\tau_2]\!])}$
$\mathbf{fix}\ g\ x := e : \tau_1 \to \tau_2$		$\delta_{(\mathbf{fix}\ g\ x:=e:\tau_1\to\tau_2,\ \mathbf{fix}\ g\ x:=\mathcal{D}[\![e]\!]:\mathcal{D}[\![\tau_1]\!]\to\mathcal{D}[\![\tau_2]\!])}$
$(e_1 : \tau_1 \to \tau_2\ \ e_2 : \tau_1) : \tau_2$	$e_1 = \mathbf{fun}\ x \to e$ or $\mathbf{fix}\ f\ x := e$, $e_2 = v_2$ $e_1 = \mathbf{fun}\ x \to e$ or $\mathbf{fix}\ f\ x := e$, $e_2 \neq v_2$ o.w.	$\delta_{(e[v_2/x]:\tau_2,\ \mathcal{D}[\![e]\!][\mathcal{D}[\![v_2]\!]/x]:\mathcal{D}[\![\tau_2]\!])}$ $[\![e_2 : \tau_1, \mathcal{D}[\![e_2 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(e_1:\tau_1\to\tau_2\ \ g:\tau_1:\tau_2,\ \mathcal{D}[\![e_1]\!]:\mathcal{D}[\![\tau_1]\!]\to\mathcal{D}[\![\tau_2]\!]\ \ g':\mathcal{D}[\![\tau_1]\!]:\mathcal{D}[\![\tau_2]\!])}$ $[\![e_1 : \tau_1 \to \tau_2, \mathcal{D}[\![e_1]\!] : \mathcal{D}[\![\tau_1 \to \tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(g\ e_2:\tau_1:\tau_2,\ g'\ \ \mathcal{D}[\![e_2]\!]:\mathcal{D}[\![\tau_1]\!]:\mathcal{D}[\![\tau_2]\!])}$
nil : $\mathbf{list}(\tau)$		$\delta_{(\text{nil}:\mathbf{list}(\tau),\ \text{nil}:\mathbf{list}(\mathcal{D}[\![\tau]\!]))}$
$(e_1 : \tau :: e_2 : \mathbf{list}(\tau)) : \mathbf{list}(\tau)$	$e_1 = v_1,\ e_2 = v_2$ $e_1 = v_1,\ e_2 \neq v_2$ o.w.	$\delta_{(v_1::v_2:\mathbf{list}(\tau),\ \mathcal{D}[\![v_1]\!]::\mathcal{D}[\![v_2]\!]:\mathbf{list}(\mathcal{D}[\![\tau]\!]))}$ $[\![e_2 : \mathbf{list}(\tau), \mathcal{D}[\![e_2]\!] : \mathbf{list}(\mathcal{D}[\![\tau]\!])]\!]$ $\gg\!= \lambda(g, g').\delta_{(v_1::g:\mathbf{list}(\tau),\ \mathcal{D}[\![v_1]\!]::g':\mathbf{list}(\mathcal{D}[\![\tau]\!]))}$ $[\![e_1 : \tau, \mathcal{D}[\![e_1]\!] : \mathcal{D}[\![\tau]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(g::e_2:\mathbf{list}(\tau):\mathbf{list}(\tau),\ g'::\mathcal{D}[\![e_2]\!]:\mathbf{list}(\mathcal{D}[\![\tau]\!]):\mathbf{list}(\mathcal{D}[\![\tau]\!]))}$
$(\mathbf{match}\ e : \mathbf{list}(\tau_1)\ \mathbf{with}$ $\quad \mid \text{nil} \to e_1 : \tau_2$ $\quad \mid h :: t \to e_2 : \tau_2$ $\quad \mathbf{end}) : \tau_2$	$e = \text{nil}$ $e = v_h :: v_t$ o.w.	$\delta_{(e_1:\tau_2,\ \mathcal{D}[\![e_1:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!])}$ $\delta_{(e_2[v_h/h, v_t/t]:\tau_2,\ \mathcal{D}[\![e_2:\tau_2]\!][\mathcal{D}[\![v_h]\!]/h, \mathcal{D}[\![v_t]\!]/t]:\mathcal{D}[\![\tau_2]\!])}$ $[\![e : \mathbf{list}(\tau_1), \mathcal{D}[\![e]\!] : \mathbf{list}(\mathcal{D}[\![\tau_1]\!])]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathbf{match}\ g\ \mathbf{with}\ \mid\text{nil}\to e_1:\tau_2\ \mid h::t\to e_2:\tau_2\ \mathbf{end}:\tau_2,$ $\qquad \mathbf{match}\ g'\ \mathbf{with}\ \mid\text{nil}\to\mathcal{D}[\![e_1:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]\ \mid h::t\to\mathcal{D}[\![e_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]\ \mathbf{end}:\mathcal{D}[\![\tau_2]\!])}$

Table 4. Small-step coupling semantics (continued). Here, every function of the form $f(g, g') = \lambda(g, g').\delta_{\ldots}$ is extended to $f_\perp$ in the sense of Definition F.29.

and equip it with the trace σ-algebra inherited from the product:

$$\mathcal{F}_{P_\tau} := \{P_\tau \cap A \mid A \in \mathcal{F}_{Expr_\tau^{\cup}} \otimes \mathcal{F}_{Expr_{\mathcal{D}[\![\tau]\!]}^{\cup+}}\}.$$

The error-preserving component maps

$$p_1 : P_\tau + \perp \to Expr_\tau^{\cup} + \perp, \qquad p_2 : P_\tau + \perp \to Expr_{\mathcal{D}[\![\tau]\!]}^{\cup+} + \perp$$

from Definition F.6 are measurable.

This trace construction does not require P_τ to be a measurable event in the ambient product and therefore does not silently assume measurability of the type-directed transformation. It also does not make P_τ closed under the displayed synchronous successor rules. In fact, the literal generated target can take deterministic administrative steps while the source stutters, and discretization need not commute syntactically with substitution. We therefore do not declare the operational coupling to take values in $\mathcal{D}(P_\tau + \bot)$.

The appropriate replacement is a measurable weak-simulation relation $R_\tau \subseteq Expr_\tau^{\cup} \times Expr_{\mathcal{D}[\![\tau]\!]}^{\cup+}$ containing every initial pair $(e, \mathcal{D}[\![e]\!])$ for every closed source term e. Such a relation must be closed under coupled successors after finite target administrative reductions.

Definition F.41 (Weak coupling interface (assumed)). Suppose R_τ has the trace σ-algebra and there is a measurable sub-Markov coupling kernel

$$K_\tau : R_\tau \longrightarrow \mathcal{D}(R_\tau + \bot)$$

whose error-preserving component pushforwards are the source one-step kernel and the target weak-step kernel, respectively. The target weak step may include finitely many deterministic administrative reductions before and after its probabilistic step. Closure, measurability, and these two pushforward identities are premises; the synchronous tables do not establish them for the literal generated target.

Lemma F.42 (Conditional coupled bind). *For every occurrence of a monadic bind $\mu \gg\!= (f, f')$ given in the small-step operational coupling semantics in Tables 3 and 4, $(f, f') : (P_{\tau_1} + \bot) \to \mathcal{D}(P_{\tau_2} + \bot)$ is a Markov kernelUnder Definition F.41, K_τ and its $\bot$-extension are sub-Markov kernels with the indicated measurable source and target spaces.*

Proof. Analogous to proof of Theorem F.38The claim for K_τ is an assumption. The error extension follows from Lemma F.30, with "sub-Markov" in place of "Markov" as in the separately stated kernel-category repair. □

Lemma F.43 (Conditional small-step coupling). *For each type τ, the small-step operational coupling semantics $[\![\cdot, \cdot]\!] : P_\tau \to \mathcal{D}(P_\tau + \bot)$ defines a Markov kernel Under Definition F.41, the weak coupled step on R_τ is a sub-Markov kernel by assumption.*

Proof. Analogous to proof of Theorem F.39This is exactly the kernel premise in Definition F.41. □

We now define suitable projection functions on measures $\mu \in \mathcal{D}(P_\tau + \bot)$ by For measures on $R_\tau + \bot$, we use the measurable projections from Definition F.6. Equivalently, $\pi_i(\mu) = \mu \gg\!= (\lambda(x_1, x_2).\delta_{x_i})$. This continuation is defined only on ordinary related pairs; strict bind supplies its canonical extension at $\bot$, which sends error to $\delta_\bot$.

We have the following key property:

Lemma F.44 (Conditional small-step coupling projection). *Let $e : \tau$. We Under Definition F.41, the component pushforwards of K_τ are the source one-step and target weak-step kernels. In particular, for each related pair $(e, t) \in R_\tau$ we have:*

(1) $\pi_1([\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![e : \tau]\!]\pi_1(K_\tau(e, t)) = [\![e : \tau]\!]$.

(2) $\pi_2([\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![\mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]\pi_2(K_\tau(e, t))$ *is the target weak-step distribution from t.*

Proof. Both identities are premises of Definition F.41. The disabled synchronous calculation above cannot establish them for the literal generated target. □

Lemma F.45. *Let $F\colon P_\tau \to \mathcal{D}(P_\tau + \bot)$, let $G\colon Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$, (X_1, Σ_{X_1}), (X_2, Σ_{X_2}), (Y_1, Σ_{Y_1}), and (Y_2, Σ_{Y_2}) be measurable spaces. Let $C_X \subseteq X_1 \times X_2$ and let $G'\colon Expr_{\mathcal{D}[\![\tau]\!]} \to \mathcal{D}(Expr_{\mathcal{D}[\![\tau]\!]} + \bot)$. $C_Y \subseteq Y_1 \times Y_2$ carry their trace σ-algebras. Let*

$$F : C_X \to \mathcal{D}(C_Y + \bot), \qquad G_i : X_i \to \mathcal{D}(Y_i + \bot)$$

be ordinary sub-Markov kernels. Every bind below is the strict bind from the kernel-category development, which performs the absorbing $\bot$-extension internally. We have:

(1) If $\forall (f, f') \in P_\tau\colon F(f, f') = G(f)\pi_1(F(x_1, x_2)) = G_1(x_1)$ for every $(x_1, x_2) \in C_X$, then for all $\mu \in \mathcal{D}(P_\tau + \bot)\mu \in \mathcal{D}(C_X + \bot)$,

$$\pi_1(\mu \gg\!= F) = \pi_1(\mu) \gg\!= G_1.$$

(2) If $\forall (f, f') \in P_\tau\colon F(f, f') = G'(f)\pi_2(F(x_1, x_2)) = G_2(x_2)$ for every $(x_1, x_2) \in C_X$, then for all $\mu \in \mathcal{D}(P_\tau + \bot)\mu \in \mathcal{D}(C_X + \bot)$,

$$\pi_2(\mu \gg\!= F) = \pi_2(\mu) \gg\!= G_2.$$

Proof. We only prove Lemma F.45.1because the reasoning for Lemma F.45.2 is completely analogous. We have ; the second part is analogous. Write p_1^X and p_1^Y for the error-preserving first coordinate maps on $C_X + \bot$ and $C_Y + \bot$. For every measurable $A \in \Sigma_{Y_1 + \bot}$, the left-hand side evaluated at A is

$$\begin{aligned}
\pi_1(\mu \gg\!= F)(A) &= (\mu \gg\!= F)((p_1^Y)^{-1}(A)) \\
&= \int F_\bot(z)((p_1^Y)^{-1}(A))\, \mu(dz) \\
&= \int (G_1)_\bot(p_1^X(z))(A)\, \mu(dz) \\
&= \int (G_1)_\bot(x_1)(A)\, \pi_1(\mu)(dx_1) \\
&= (\pi_1(\mu) \gg\!= G_1)(A).
\end{aligned}$$

Here $F_\bot$ and $(G_1)_\bot$ denote the internal error extensions made by strict bind. The third equality uses the componentwise premise when $z \neq \bot$ and the fact that both $F_\bot$ and $(G_1)_\bot \circ p_1^X$ send $\bot$ to $\delta_\bot$. The fourth equality is the defining integration identity for pushforward measures. Since A was arbitrary, the measures are equal. □

F.6 Big-Step Semantics and Correctness of Discretization

We extend both of the the ordinary small-step semantics canonically to an n-step semanticsby. Conditionally on Definition F.41, the second line denotes iteration of K_τ from the initial related pair:

$$\langle e : \tau \rangle_n = \begin{cases} \delta_{e:\tau} & \text{if } n = 0 \\ \langle e : \tau \rangle_{n-1} \gg\!= \lambda e'.[\![e' : \tau]\!] & \text{if } n > 0\,. \end{cases}$$

$$\langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!] \rangle_n = \begin{cases} \delta_{(e:\tau, \mathcal{D}[\![e]\!]:\mathcal{D}[\![\tau]\!])} & \text{if } n = 0 \\ \langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!] \rangle_{n-1} \gg\!= K_\tau & \text{if } n > 0\,. \end{cases}$$

As a sanity check, we show that the n-step semantics defines a Markov sub-Markov kernel.

Lemma F.46 (n-step semantics is a sub-Markov kernel). *For all $n \in \mathbb{N}$, $\langle \cdot \rangle_n : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov sub-Markov kernel.*

Proof. By induction on n.

Base case: $n = 0$. By definition,

$$\langle e : \tau \rangle_n = \delta_{e:\tau}$$

i.e. the map $e \mapsto \delta_e$. By Lemma F.31, this is a Markov kernel and hence a sub-Markov kernel.

Inductive step: $n > 0$. By definition,

$$\langle e : \tau \rangle_n = \langle e : \tau \rangle_{n-1} \gg\!= \lambda e'.[\![e' : \tau]\!]$$

By the I.H., $\langle \cdot \rangle_{n-1} : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov kernel . By Theorem F.39, $\lambda e'.[\![e' : \tau]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov kernel. By Lemma F.30, its $\bot$-extension from $(Expr_\tau + \bot)$ to $\mathcal{D}(Expr_\tau + \bot)$ is also a Markov kernel. By Lemma F.33, $\langle e : \tau \rangle_{n-1} \gg\!= \lambda e'.[\![e' : \tau]\!]$ is a Markov kernel. the finite-step kernel is sub-Markov. By Theorem F.39, the ordinary-input continuation is sub-Markov, and Lemmas F.30 and F.33 show that strict composition is sub-Markov. □

Lemma F.47 (Conditional n-step coupling kernel). *For all $n \in \mathbb{N}$, $\langle \cdot, \cdot \rangle_n : P_\tau \to \mathcal{D}(P_\tau + \bot)$ is a Markov kernel Under Definition F.41, for all $n \in \mathbb{N}$ the n-step weak coupling on R_τ is a sub-Markov kernel into $\mathcal{D}(R_\tau + \bot)$.*

Proof. Analogous to proof of Lemma F.46By induction on n, using closure of sub-Markov kernels under composition. □

For every type τ, we denote by $Vals_\tau$ the set of values of type τ and equip it with a suitable measurable sub-space of the measurable space over $Expr_\tau + \bot$ the trace σ-algebra inherited from $Expr_\tau$ (cf. Lemma F.27.1). Moreover, let $V_\tau = \{(v : \tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau\}$. Conditionally, let

$$V_\tau := R_\tau \cap (Vals_\tau^{\cup} \times Vals_{\mathcal{D}[\![\tau]\!]}^{\cup +})$$

with the trace σ-algebra. This is a measurable subspace because both value sets are measurable.

Lemma F.48 (Basic Properties of $\langle \cdot \rangle_n$). *Let $e : \tau \in Expr_\tau$ and $n \in \mathbb{N}$. Let $Vals_\tau$ be the measurable set of values. We have:*

(1) $\langle e : \tau \rangle_n(Vals_\tau) \le \langle e : \tau \rangle_{n+1}(Vals_\tau)$
(2) $\langle e : \tau \rangle_n(\bot) \le \langle e : \tau \rangle_{n+1}(\bot)$

Proof. By induction on n.
Base case: $n = 0$.
We must show $\langle e : \tau \rangle_0(Vals_\tau) \le \langle e : \tau \rangle_1(Vals_\tau)$.
By definition,

$$\langle e : \tau \rangle_0 = \delta_{e:\tau}.$$

Therefore,

$$\langle e : \tau \rangle_0(Vals_\tau) = \begin{cases} 1 & \text{if } e \in Vals_\tau \\ 0 & \text{if } e \notin Vals_\tau \end{cases}$$

Now, for $n = 1$, by definition,

$$\langle e : \tau \rangle_1 = \delta_{e:\tau} \gg\!= \lambda e'.[\![e']\!] = [\![e : \tau]\!].$$

We analyze the two cases for e:
Case 1: e is a value v. Then, $\langle e : \tau \rangle_0(Vals_\tau) = 1$.
For $n = 1$:

$$\langle e : \tau \rangle_1 = \delta_{v:\tau} \gg\!= \lambda e'.[\![e']\!] = [\![v : \tau]\!].$$

From the semantics, $[\![v : \tau]\!] = \delta_{v:\tau}$, which means $\langle e : \tau \rangle_1 = \delta_{v:\tau}$, and thus

$$\langle e : \tau \rangle_1(V) = 1,$$

and thus $\langle e : \tau \rangle_0(Vals_\tau) \le \langle e : \tau \rangle_1(Vals_\tau)$.
Case 2: e is an expression. Then, $\langle e : \tau \rangle_0(Vals_\tau) = 0$.
For $n = 1$:

$$\langle e : \tau \rangle_1 = \delta_{e:\tau} \gg\!= \lambda e'.[\![e']\!] = [\![e : \tau]\!].$$

From the semantics, $[\![e : \tau]\!]$ may step to either a value, which means $[\![e : \tau]\!](Vals_\tau) \geq 0$, or an expression, which means $[\![e : \tau]\!](Vals_\tau) = 0$. In either case,

$$\langle e : \tau\rangle_1(Vals_\tau) \geq 0,$$

and thus $\langle e : \tau\rangle_0(Vals_\tau) \leq \langle e : \tau\rangle_1(Vals_\tau)$.

Inductive step: We want to show that for $n = k + 1$:

$$\langle e : \tau\rangle_{k+1}(Vals_\tau) \leq \langle e : \tau\rangle_{k+2}(Vals_\tau).$$

By the recursive definition:

$$\begin{aligned}
&\langle e : \tau\rangle_k,\\
&\langle e : \tau\rangle_{k+1} = \langle e : \tau\rangle_k \gg\!= \lambda e'.[\![e']\!],\\
&\langle e : \tau\rangle_{k+2} = \langle e : \tau\rangle_{k+1} \gg\!= \lambda e''.[\![e'']\!] = (\langle e : \tau\rangle_k \gg\!= \lambda e'.[\![e']\!]) \gg\!= \lambda e''.[\![e'']\!]
\end{aligned}$$

By the IH,

$$\langle e : \tau\rangle_k(Vals_\tau) \leq \langle e : \tau\rangle_{k+1}(Vals_\tau).$$

We do a case analysis on e' and e'', whether they are in N_τ or $Vals_\tau$.

Case $e' \in Vals_\tau$, $e'' \in Vals_\tau$.

Then, from the semantics, $[\![e']\!]$ and $[\![e'']\!]$ are both Diracs over a value. Combined with the IH and the fact that $[\![\cdot]\!]$ does not change the probability mass of $\cdot$ if it is a value,

$$\langle e : \tau\rangle_{k+1}(Vals_\tau) \leq \langle e : \tau\rangle_{k+2}(Vals_\tau).$$

Case $e' \in Vals_\tau$, $e'' \in N_\tau$.

Then, from the semantics, $[\![e']\!]$ is a Dirac over a value and $[\![e'']\!]$ is a distribution over a non-value expression. Combined with the IH and the fact that $[\![\cdot]\!]$ does not change the probability mass of $\cdot$ if it is a value and that $[\![\cdot]\!]$ does not add any probability mass to $Vals_\tau$ if $\cdot$ is a non-value expression,

$$\langle e : \tau\rangle_{k+1}(Vals_\tau) \leq \langle e : \tau\rangle_{k+2}(Vals_\tau).$$

Case $e' \in Vals_\tau$, $e'' \in N_\tau$.

Analogous to the previous case, but with e' and e'' switched.

Case $e' \in N_\tau$, $e'' \in N_\tau$.

Then, from the semantics, $[\![e']\!]$ and $[\![e'']\!]$ are both distributions over a non-value expression. Combined with the IH and the fact that $[\![\cdot]\!]$ does not add any probability mass to $Vals_\tau$ if $\cdot$ is a non-value expression,

$$\langle e : \tau\rangle_{k+1}(Vals_\tau) \leq \langle e : \tau\rangle_{k+2}(Vals_\tau).$$

In all cases, $\langle e : \tau\rangle_{k+1}(Vals_\tau) \leq \langle e : \tau\rangle_{k+2}(Vals_\tau)$. □

Definition F.49. Let $e : \tau$. The *(big-step) semantics* $\langle e : \tau\rangle \in \mathcal{D}(Vals_\tau + \bot)$ and $\langle e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]\rangle \in \mathcal{D}(V_\tau + \bot)$ are respectively defined as be closed and well typed, let $T_\tau = Vals_\tau + \bot$, and let ν_n^e be the restriction of $\langle e : \tau\rangle_n$ to the measurable terminal subspace T_τ. We use the ordinary terminal-limit semantics $\langle e : \tau\rangle \in \mathcal{D}(T_\tau)$ defined by

$$\langle e : \tau\rangle(A) = \lim_{n\to\infty} \nu_n^e(A), \qquad A \in \mathcal{F}_{T_\tau}.$$

By Items 1 and 2, the sequence $\langle e : \tau\rangle_n$ is monotonically increasingfor values and error. Moreover, $\langle e : \tau\rangle_n$ defines a subprobability measure, and thus $\langle e : \tau\rangle_n \leq 1$ for every $n \in \mathbb{N}$. Thus, by the monotone convergence theorem, the pointwise limit exists. The separate ordinary-semantics development proves that $(\nu_n^e)_n$ is increasing on every measurable terminal event and that the pointwise limit is a subprobability measure. The full measures on expression states are not claimed to be increasing.

The weak coupling interface alone does not imply that restriction of its iterates to terminal related pairs is increasing. We isolate the additional fact needed for a coupled big-step conclusion.

Definition F.50 (Assumption: terminal weak-coupling limit). Under Definition F.41, suppose the restrictions of the iterates of K_τ from $(e, \mathcal{D}[\![e]\!])$ to $V_\tau + \bot$ form an increasing sequence $(\rho_n)_n$ with a subprobability setwise limit ρ_∞. Suppose its first component restrictions agree with the ordinary source terminal restrictions, and suppose the limit of its second component restrictions equals the ordinary target terminal limit. The latter is a limit-compatibility premise, not a same-index finite-step identity: a target weak step may contain several ordinary administrative steps. Finally, suppose related Boolean values have the same truth value.

Lemma F.51 (Projection commutes with increasing limits). *Let $(\rho_n)_n$ be an increasing sequence of subprobability measures on a measurable relation $R + \bot$, and suppose $\rho_\infty(E) = \lim_n \rho_n(E)$ is its setwise limit. For either error-preserving component map p_i and every measurable component event A,*

$$(p_i)_*\rho_\infty(A) = \lim_n (p_i)_*\rho_n(A).$$

Proof. The preimage $p_i^{-1}(A)$ is measurable, so

$$(p_i)_*\rho_\infty(A) = \rho_\infty(p_i^{-1}(A)) = \lim_n \rho_n(p_i^{-1}(A)) = \lim_n (p_i)_*\rho_n(A).$$

□

Lemma F.52 (Conditional n-step coupling projection). *For all $n \in \mathbb{N}$Under Definition F.41, let W_τ be the target weak-step kernel and let $W_\tau^{(n)}$ denote its n-fold iteration. For all $n \in \mathbb{N}$,*

(1) $\pi_1(\langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]\rangle_n) = \langle e : \tau\rangle_n$

(2) $\pi_2(\langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]\rangle_n) = \langle \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]\rangle_n$ $\pi_2(\langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]\rangle_n) = W_\tau^{(n)}(\mathcal{D}[\![e]\!])$.

Proof. By induction on n. We prove i), as the proof for ii) is analogousFor $n = 0$, the result follows because $\pi_i(\delta_{(e,\mathcal{D}[\![e]\!])})$ is the Dirac measure at the corresponding coordinate.

Base case: $n = 0$ For the inductive step, put $K(f, g) = K_\tau(f, g)$, $G_1(f) = [\![f : \tau]\!]$, and $G_2(g)$ equal the target weak-step kernel. By Lemma F.44, $\pi_i(K(f, g)) = G_i(p_i(f, g))$. Therefore Lemma F.45 gives

$$\pi_i(\langle e, \mathcal{D}[\![e]\!]\rangle_{n-1} \gg\!= K) = \pi_i(\langle e, \mathcal{D}[\![e]\!]\rangle_{n-1}) \gg\!= G_i.$$

Inductive step: $n > 0$ Applying the induction hypothesis and the definition of the n-step semantics proves the first result for $i = 1$ and the second for $i = 2$. □

Lemma F.53 (Conditional big-step coupling projection). *For a closed, well-typed $e : \tau$, under Definitions F.41 and F.50, the terminal coupling limit ρ_∞ has projections*

$$\pi_1(\rho_\infty) = \langle e : \tau\rangle, \qquad \pi_2(\rho_\infty) = \langle \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]\rangle.$$

Proof. Apply Lemma F.51 to each component. The source finite terminal-restriction identity and the target limit-compatibility identity are premises of Definition F.50. Their limits are the two ordinary big-step semantics. □

Theorem F.54 (Conditional Boolean outcome preservation). *Under Definitions F.41 and F.50, if e : **bool** is closed and well typed, then $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e]\!] : \mathbf{bool}\rangle$.*

Proof. For $b \in \{\text{true}, \text{false}\}$, the premise that related Boolean values have the same truth value gives $p_1^{-1}(\{b\}) = p_2^{-1}(\{b\})$ inside $V_{\mathbf{bool}} + \bot$. Both error-preserving projections also have preimage $\{\bot\}$ at $\bot$. Hence the two projections of ρ_∞ agree on true, false, and $\bot$. They are the displayed ordinary semantics by Lemma F.53. □